LeakSense – Smart LPG Gas Leakage Detection and Automatic Safety System

CITS5506 The Internet of Things
The University of Western Australia, Australia
Semester 1, 2026

Group 27

1st Yuxuan Xi, 24319908@student.uwa.edu.au
2nd Yiming Zhao, 24139958@student.uwa.edu.au;
3rd Sahil Pankajbhai Patel, 24562775@student.uwa.edu.au;
4th Sohaib Ahmed Khan, 24756957@student.uwa.edu.au

Abstract

Residential LPG leakage is a serious household safety problem because accumulated gas can cause fire, explosion, and health hazards, yet affordable detectors typically provide only local alarms without physical mitigation or remote monitoring. This report presents LeakSense, a low-cost Internet of Things system that detects LPG risk, responds locally, and provides real-time cloud visibility. The system was developed using an iterative development method and integrates an ESP32 microcontroller, a MEMS gas sensor, a BME280 atmospheric sensor, relay-driven actuators, Firebase Realtime Database, and a browser dashboard. The firmware samples gas voltage every two seconds and classifies readings into Safe, Warning, Danger, and Extreme states, escalating Warning to Danger when abnormal temperature rise or high absolute temperature indicates additional thermal risk. Local actuators operate independently of network connectivity, while Firebase uploads support live status, history logging, and browser notifications. Testing confirmed correct state classification across all thresholds, actuator response within the two-second polling interval, cloud-to-dashboard updates within three seconds, successful push notifications, and continued local safety operation during Wi-Fi interruption. LeakSense cost approximately $94 AUD. Key limitations are the small-scale fan, the absence of an automatic gas shutoff valve, and empirically determined voltage thresholds that require formal calibration against certified reference gas for production deployment.

Keywords

Internet of Things; LPG gas detection; ESP32; Firebase; dual-sensor escalation; residential safety; web dashboard

I. Problem Statement

LPG is used in over two million Australian households for cooking, heating, and hot water systems [1]. In its natural state, LPG is odourless and can accumulate to dangerous concentrations before occupants become aware of a leak. According to Fire and Rescue NSW, gas-related incidents account for a significant proportion of residential fire call-outs each year [2].

Existing residential gas detection solutions fail to address the full scope of this safety problem. Basic standalone detectors produce only a local audible alarm, which is ineffective when occupants are asleep, absent, or hearing-impaired, and do not initiate any physical hazard mitigation. Smart consumer detectors provide mobile push notifications but remain proprietary, do not control physical ventilation, and do not expose sensor data for custom analysis. Research-grade IoT prototypes have demonstrated remote alerting but typically use analog sensors without environmental compensation, rely on single-sensor detection, and lack multi-threshold response logic and cloud-connected dashboards [3]. Industrial detection systems provide certified shutoff valve integration and high measurement accuracy but are prohibitively expensive for residential installation.

A critical and unaddressed limitation across all affordable solutions is the absence of multi-factor escalation. Gas leakage in combination with a sudden rise in ambient temperature — which may indicate an ignition or fire risk — is not modelled in any existing consumer product. The central problem is therefore the absence of an affordable, integrated residential solution that combines: continuous gas voltage monitoring, graduated multi-threshold response with thermal escalation, automatic physical

hazard mitigation, and remote cloud-based monitoring with historical data logging. LeakSense is designed specifically to fill this gap.

II. Introduction

The Internet of Things enables physical sensor systems to communicate with cloud infrastructure and user-facing applications, creating safety systems that respond both locally and remotely. LeakSense applies this IoT architecture to the practically important problem of residential LPG gas leakage detection, extending prior single-sensor approaches with a dual-sensor escalation model that integrates gas concentration voltage with ambient temperature readings.

Prior work in this area falls into four broad categories. First, passive standalone detectors provide local alarms but no remote awareness or physical response. Second, smart consumer products add cloud notification but maintain proprietary data ecosystems that prevent custom integration. Third, academic IoT prototypes demonstrate cloud-connected gas detection using commodity microcontrollers but typically rely on single analog MQ-series sensors without environmental compensation [3] [4]. Fourth, industrial systems provide the most complete safety response but at a scale and cost entirely unsuitable for residential use. A consistent limitation across affordable solutions is the absence of automatic ventilation control, thermal escalation logic, and real-time web-based monitoring with historical trend data [12].

This report describes the design, implementation, and testing of LeakSense — a system that addresses the identified gaps using a FireBeetle ESP32-E microcontroller, a SEN0565 MEMS analog gas sensor, a BME280 atmospheric sensor, relay-controlled actuators, Firebase Realtime Database via HTTPS REST, and a plain HTML/CSS/JavaScript web dashboard. The system was designed using an iterative prototyping methodology. Success was defined as correct state classification across all thresholds, actuator response within two seconds, end-to-end cloud-to-dashboard latency within three seconds, and continued local safety operation during Wi-Fi interruption.

The remainder of this report is structured as follows. Section III describes the system architecture and hardware design. Section IV presents the firmware implementation including thermal escalation logic. Section V describes the web dashboard and cloud architecture. Section VI presents testing methodology and results. Section VII discusses limitations and future work. Section VIII presents the project cost. Section IX concludes the report. Section X provides instructions for hardware setup, firmware installation, Firebase configuration, and dashboard operation. The references follow the How-To Guide, and the appendices provide the code function explanations and source code listings.

III. System Architecture

A. Overview

LeakSense is structured around five IoT layers: sensing, computing, actuation, cloud communication, and user interface. The block diagram in Fig. 1 illustrates the five subsystems, their data flows, and their interconnections. Two distinct data paths exist: a local safety path from sensing through processing to actuation that operates continuously regardless of network availability, and a cloud path from processing through Firebase to the dashboard that operates on a best-effort basis.

The ESP32 is the central integration point. It polls the gas sensor and BME280 every two seconds, applies a two-pass state classification algorithm (voltage thresholds followed by a thermal escalation check), drives all local actuators independently of network availability, and publishes data to Firebase every two seconds for the live dashboard and every five minutes for historical records.

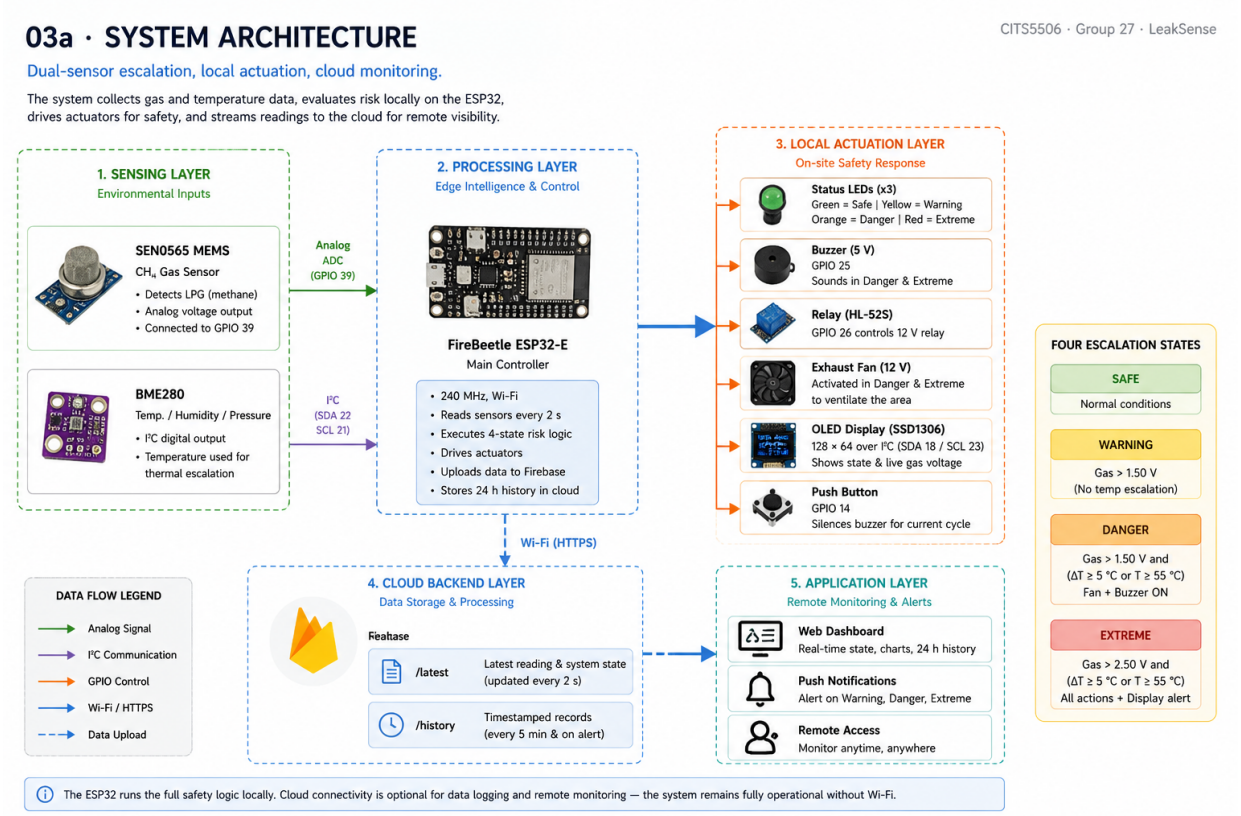

Fig. 1. LeakSense system block diagram showing all five IoT layers and their interconnections. (REST = Representational State Transfer; HTTPS = Hypertext Transfer Protocol Secure; LED = light-emitting diode; OLED = organic light-emitting diode.)

B. Component Overview

Table I lists all hardware components used in the LeakSense system.

TABLE I
LEAKSENSE HARDWARE COMPONENTS AND GPIO ASSIGNMENTS.

Component	Purpose	GPIO	Bus
FireBeetle ESP32-E	Central MCU — Wi-Fi, I2C, GPIO, 240 MHz dual-core	—	—
SEN0565 MEMS CH4	Analog gas voltage — primary input	39	ADC
BME280	Temp (escalation) + Humidity (display)	22/21	I2C-1
OLED SSD1306 128×64	Local real-time status display	18/23	I2C-0
HL-52S Relay	Controls exhaust fan	26	GPIO
12 V Exhaust Fan	Ventilation via relay	(relay)	—
5 V Buzzer	Audible alarm — 10 s cycle	25	GPIO
Green LED	Safe state indicator	17	GPIO
Yellow LED	Warning state indicator	16	GPIO
Red LED	Danger/Extreme indicator	4	GPIO
Push Button	Silences current buzzer cycle	14	GPIO

C. Component Descriptions

1) FireBeetle ESP32-E: The FireBeetle ESP32-E was selected because it integrates Wi-Fi, two I2C buses, ADC-capable GPIO, and a dual-core 240 MHz processor on one compact board. It is fully compatible with PlatformIO and the Arduino framework, which reduces firmware development time. The built-in Wi-Fi module enables direct HTTPS REST communication with Firebase Realtime Database without an additional networking component [9].The FireBeetle ESP32-E board is shown in Fig. 2.

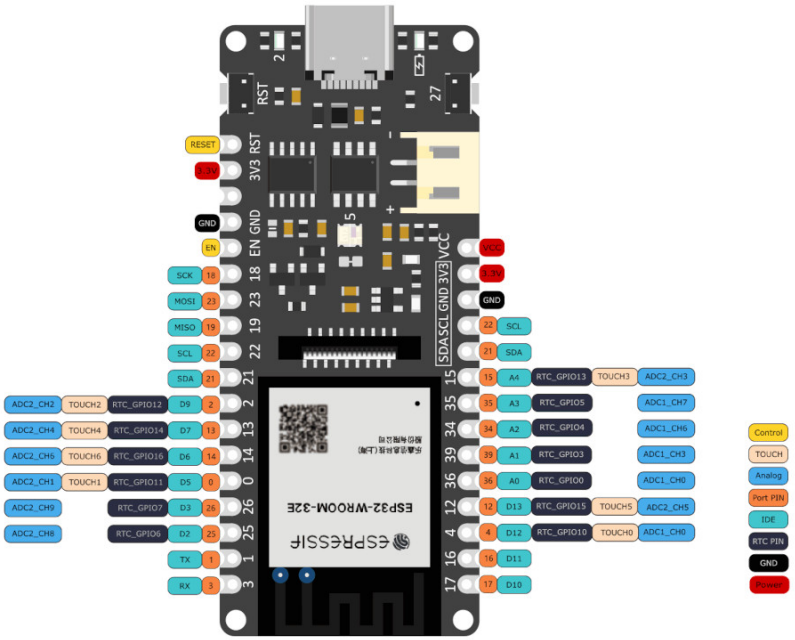

Fig. 2. FireBeetle ESP32-E Microcontroller serving as the central processing unit, responsible for sensor polling, state classification, actuator control, and cloud communication.

2) SEN0565 MEMS CH4 Gas Sensor: The SEN0565 Fermion MEMS CH4 sensor was selected as the primary gas detection input. The sensor outputs an analog voltage on GPIO 39 that rises with increasing gas concentration, without requiring manual Rs/R0 resistance ratio conversion or sensitivity curve calculation. DFRobot documentation confirms the sensor is intended for qualitative measurement only and does not output calibrated concentration values in parts per million [5]. The sensor requires a warm-up period of at least five minutes before readings are stable, and 24 hours if it has been unused for an extended period [6].

Because no manufacturer-certified LPG-to-voltage mapping exists for this sensor, voltage thresholds were determined empirically by observing serial monitor output during controlled lighter-gas exposure in a laboratory environment. The resulting thresholds — 1.60 V for Warning and 1.90 V for Danger — are empirical calibration constants. For a production safety system, formal calibration against certified LPG reference gas or a known percentage lower-explosive-limit (LEL) standard would be required. The OSHA chemical data for LPG states that the LEL for propane is approximately 2.1% by volume and 1.9% for butane [7], which would serve as the reference concentration targets for proper calibration.The SEN0565 sensor module is shown in Fig. 3.

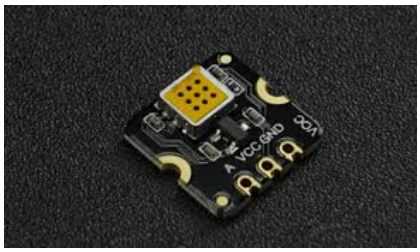

Fig. 3. SEN0565 MEMS CH4 Sensor providing an analog voltage output proportional to gas concentration, used as the primary input for state classification.

3) BME280 Atmospheric Sensor: The BME280 provides ambient temperature and humidity measurements over I2C on a dedicated bus (SDA GPIO 22, SCL GPIO 21) [10]. Temperature serves as a secondary escalation input: if the temperature rises 5 °C or more between consecutive two-second readings, or reaches 55 °C absolute, and the gas state is already Warning, the system escalates immediately to Danger and records `thermal_risk: true` in the Firebase payload. Humidity is displayed on the dashboard for environmental context and has no effect on the safety state. This dual-sensor approach is supported by Wu et al., who demonstrated that combining gas and temperature measurements significantly reduces false positive alarm rates compared to single-sensor designs [8].The BME280 module is shown in Fig. 4.

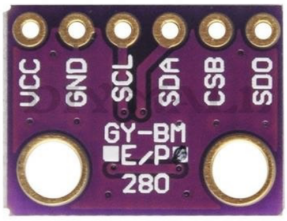

Fig. 4. BME280 Atmospheric Sensor providing temperature and humidity readings for thermal escalation and dashboard display.

4) OLED Display SSD1306: The SSD1306 0.96-inch I2C OLED display provides the local visual interface for the device. It was selected because it operates from 3.3 V logic and power, which is directly compatible with the ESP32 without level-shifting, and because its small footprint suits a compact enclosure [13]. The display is connected to a dedicated I2C bus (TwoWire 0) on SDA GPIO 18 and SCL GPIO 23 at address 0x3C, separated from the BME280 bus to avoid address contention.

During operation the display shows the current temperature, humidity, gas voltage, baseline voltage, classified state (Safe, Warning, or Danger), and the on/off status of the fan and buzzer, as drawn by the `drawStatusScreen()` routine in Section IV. This makes the device usable as a standalone unit during demonstrations without opening the web dashboard, and it accelerated firmware debugging during development because sensor readings and state transitions could be observed directly on the device. The OLED module is shown in Fig. 5.

Fig. 5. 0.96-inch SSD1306 I2C OLED display providing local status output, showing live temperature, humidity, gas voltage, and classified safety state.

5) Buzzer and LED Indicators: The buzzer provides the audible local alarm when the system enters the Danger state. A Gravity digital buzzer module was selected because it is driven directly from a single GPIO line — it accepts a HIGH/LOW logic signal from the ESP32 and requires no PWM generation or external driver circuitry [14]. It is connected to GPIO 25 and operates on the ten-second on/off cycle described in Section IV.

Three through-hole LEDs provide additional visual status indication independent of the OLED: green (GPIO 17) for the Safe state, yellow (GPIO 16) for Warning, and red (GPIO 4) for Danger and Extreme. Each LED is wired through a 270 Ω current-limiting resistor to ground to keep forward current within the safe operating range for standard 5 mm LEDs at 3.3 V logic level. The three-colour scheme allows the system state to be identified at a glance from across a room, complementing the more detailed OLED readout. The buzzer module is shown in Fig. 6.

Fig. 6. Gravity digital buzzer module providing the audible alarm during Danger and Extreme states, driven from GPIO 25.

6) Relay Module and Exhaust Fan: The HL-52S 5 V single-channel relay module provides the switching interface between the ESP32 and the exhaust fan. The ESP32 cannot directly drive motor-class loads from its GPIO pins, so the relay isolates the microcontroller from the fan's switching transients while allowing a logic-level signal to control the higher-current path. The HL-52S is rated to switch up to 10 A at 28 VDC and draws approximately 60 mA at 5 V during activation, well within the project power budget [15]. It is driven from GPIO 26 with active-low polarity matching the module design.

The Delta 5 V DC frameless exhaust fan (40 mm, ∼0.1 A, ∼3150 RPM) is the project's physical mitigation device [16]. When the gas concentration crosses the Danger threshold, the relay closes the fan supply circuit and the fan disperses accumulated gas in the monitored space. This is the design choice that elevates LeakSense from a passive alarm system to an active safety system — the fan provides immediate hazard mitigation without waiting for occupant intervention. The 40 mm form factor is appropriate for laboratory demonstration; a production deployment would substitute a mains-rated exhaust fan or integrate with existing kitchen ventilation, as discussed in Section VII. The fan is shown in Fig. 7.

Fig. 7. Delta 5 V DC 40 mm frameless exhaust fan, switched via the HL-52S relay on GPIO 26 to disperse accumulated gas during Danger and Extreme states.

7) Supporting Assembly Components: The remaining components support wiring, assembly, and stable testing of LeakSense. An 830 tie-point solderless breadboard with dual isolated power rails was used as the development platform because it provides sufficient connection density for the seven sensor and actuator modules without requiring soldered interconnects [17]. Male-to-male and male-to-female jumper wires route signals between the ESP32 and the sensor and output modules, with male-to-female ends used for the sensor and OLED headers that expose female pin sockets. A USB-A to USB-C cable supplies 5 V power to the FireBeetle ESP32-E and provides the serial programming and debugging interface to the development host. A push button on GPIO 14, configured with the ESP32 internal pull-up resistor, allows the operator to silence the current buzzer cycle as described in Section IV. A small project enclosure was used during testing to simulate a partially enclosed residential space in which leaked gas can accumulate to a detectable concentration around the SEN0565 sensing element.

D. Circuit Wiring

Two separate I2C buses are used. Bus 0 (TwoWire 0) operates on SDA GPIO 18 and SCL GPIO 23, serving the OLED SSD1306 display at I2C address 0x3C. Bus 1 (TwoWire 1) operates on SDA GPIO 22 and SCL GPIO 21, serving the BME280 at address 0x76 or 0x77. This separation prevents address conflicts. The SEN0565 analog output connects to GPIO 39, configured as a 12-bit ADC input with 11 dB attenuation for a full 0–3.3 V measurement range.

The relay module is driven from GPIO 26 (active-low, matching the HL-52S module polarity) and switches the exhaust fan's 12 V supply rail. The buzzer is driven from GPIO 25. Green, yellow, and red LEDs connect through 270 Ω current-limiting resistors to GPIO 17, GPIO 16, and GPIO 4 respectively. The push button connects GPIO 14 to ground with the ESP32 internal pull-up resistor enabled.

Fig. 8 shows the complete electrical setup used for the LeakSense. It identifies the ESP32 GPIO assignments, the two separate I2C buses, the SEN0565 analog input path, the LED indicator outputs, the alarm button input, and the relay-controlled fan and buzzer structure. This diagram therefore acts as the wiring reference for how the hardware subsystems are connected to the microcontroller.

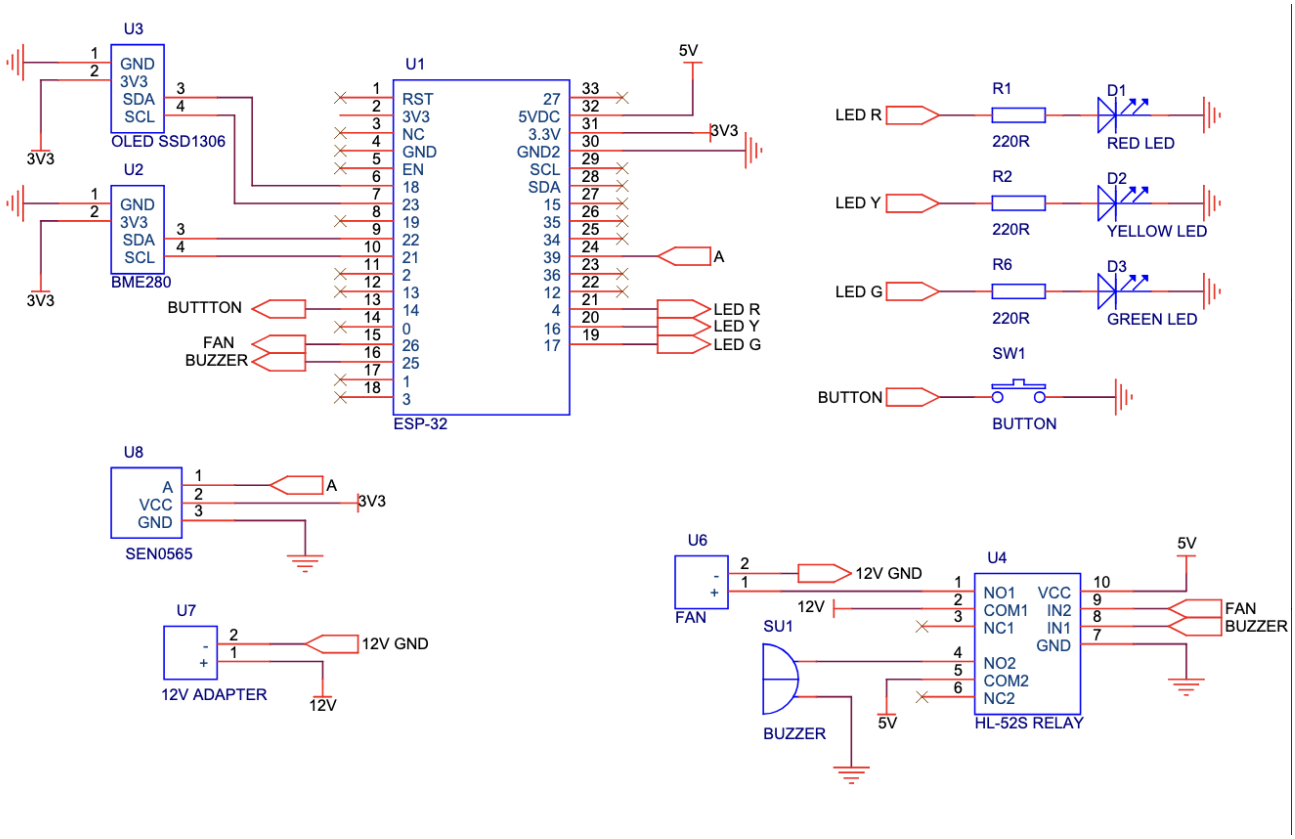

Fig. 8. LeakSense electronic schematic showing the ESP32 pin assignments, dual I2C sensor buses, SEN0565 analog input, LED indicators, push button, relay module, buzzer, fan, and 12 V/5 V power connections.

Fig. 9 is the physical representation of the same circuit on the breadboard. It shows how the schematic connections were implemented in the system, including the ESP32-E placement, sensor modules, OLED display, relay module, fan, buzzer, LEDs, and jumper-wire routing used during testing.

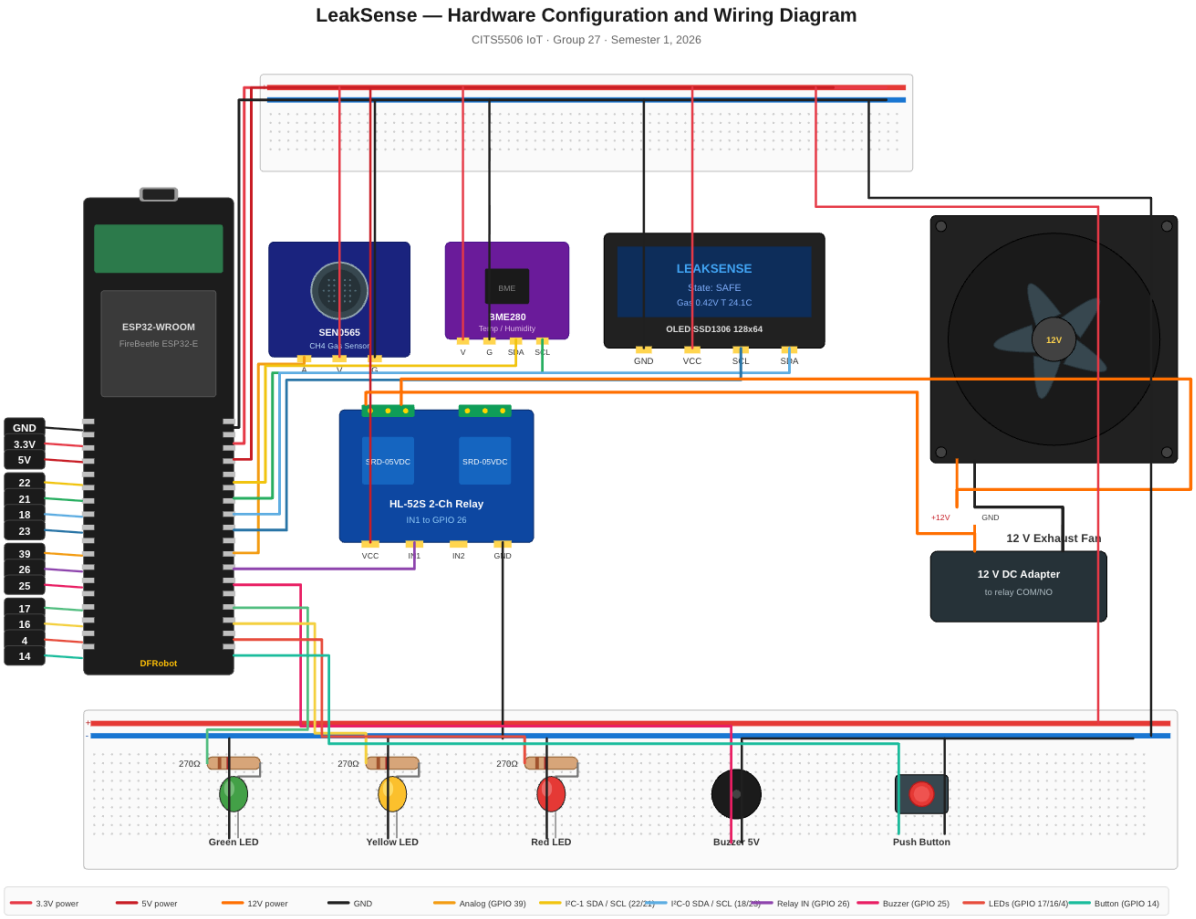

Fig. 9. LeakSense breadboard circuit showing the ESP32-E, SEN0565, BME280, OLED, relay module, fan, buzzer, and LEDs.

IV. Firmware Implementation

A. Design Approach

The firmware was developed in PlatformIO using the Arduino framework and an iterative prototyping methodology. Each subsystem — I2C sensors, state machine, thermal escalation, and Firebase REST publishing — was verified independently before the next dependency was added. This approach ensured working baselines at each stage and minimised debugging complexity during integration.

B. Boot Sequence

On power-up the firmware executes six initialisation steps:

1) All GPIO pins are set to safe defaults: green LED on, all alarm actuators off, relay off.
2) A relay self-test fires for 500 ms to confirm hardware continuity.
3) I2C bus 0 is initialised and scanned, followed by OLED display initialisation at address 0x3C.
4) I2C bus 1 is initialised and scanned, followed by BME280 initialisation at address 0x76/0x77.
5) The SEN0565 ADC input is configured on GPIO 39 with 12-bit resolution and 11 dB attenuation. A ten-second baseline period is enforced with a serial monitor warning to allow sensor pre-heating. The manufacturer recommends at least five minutes warm-up [6].
6) Wi-Fi connection is attempted with a ten-second timeout, followed by Firebase initialisation if connected. If Wi-Fi fails, the system continues in offline mode with all local safety functions active.

C. State Classification

State classification is a two-pass algorithm executed on every two-second polling cycle. The first pass derives the base state from the gas sensor voltage reading using fixed thresholds. The second pass applies the thermal escalation check. The firmware defines three hardware states; Table II defines the complete state-to-action mapping including the Extreme dashboard display label.

TABLE II
STATE MACHINE: VOLTAGE THRESHOLDS, FIRMWARE STATES, DASHBOARD LABELS, AND SYSTEM RESPONSES.

Voltage	Firmware	Dashboard	Hardware	Actions
< 1.60 V	SAFE	Safe	Green LED	Fan off · Buzzer off · Firebase updated every 2 s
1.60–1.89 V	WARNING	Warning	Yellow LED	Fan off · Buzzer off · History logged immediately
≥ 1.90 V	DANGER	Danger	Red LED	Fan ON via relay · Buzzer 10 s cycle · Firebase alert
≥ 2.20 V	DANGER*	Extreme*	Red LED	Identical to Danger. Extreme is a dashboard display label only.

**Note: Extreme is a dashboard-only display label applied when voltage exceeds 2.20 V. The firmware hardware response is identical to Danger — same red LED, same fan activation, same buzzer cycle. The Extreme label provides the remote user with an immediately recognisable indication that the reading is critically elevated. No additional firmware state or actuator behaviour is introduced.*

D. Thermal Escalation Logic

The thermal escalation check runs as the second pass of state classification, after the voltage-based base state has been determined. Two conditions can promote a Warning base state to Danger:

- The temperature reading has risen by 5 °C or more since the previous two-second polling cycle (indicating a sudden thermal event).
- The absolute temperature has reached or exceeded 55 °C (indicating sustained extreme heat).

If either condition is met and the base state is Warning, the final state is promoted to Danger and the Firebase payload includes the field `thermal_risk: true`. This allows the dashboard and alert log to distinguish a pure gas event from a combined gas-and-heat event, which may indicate ignition or fire risk. Temperature alone — with the gas voltage in the Safe range — never triggers the alarm.

A worked example from testing: gas voltage reads 1.70 V (Warning base state); temperature rises from 25.0 °C to 31.0 °C between readings (a 6.0 °C rise, exceeding the 5 °C threshold); thermal escalation fires; final state is Danger; Firebase receives {`"state": "DANGER", "thermal_risk": true`}.

E. Actuator Control

Actuator states are determined by the final classified state and applied on every polling cycle. In the Safe state, the green LED is active and all alarm actuators are deactivated. In Warning, the yellow LED is active; the fan and buzzer remain off, as Warning represents elevated concentration without immediate danger. In Danger or Extreme, the red LED is active, the relay drives the exhaust fan, and the buzzer starts a ten-second alarm cycle.

The buzzer operates on a ten-second on/off cycle managed by `millis()`-based timing. When the buzzer completes a ten-second active period, it checks the current state: if still Danger, a new cycle begins automatically. The push button on GPIO 14 silences the current cycle only — it sets a silence flag that suppresses the buzzer output until the next cycle boundary, at which point the silence flag resets and the buzzer restarts if the state remains Danger. A 50 ms debounce is applied to the button input. Actuator states are reapplied on every polling cycle rather than only on state transitions, ensuring that outputs can never become stuck in an incorrect state.

F. Cloud Communication

The ESP32 connects to the local Wi-Fi network using credentials stored in `firmware/blinking/src/secrets.h`. Firebase communication uses direct HTTPS REST calls — no third-party Firebase library is used, eliminating dependency version conflicts on the ESP32 toolchain.

A `PUT` request is made to `/leaksense/latest.json` every two seconds, overwriting the existing document so the dashboard always shows the current reading [11]. A `POST` request is made to `/leaksense/history.json` every five minutes and immediately on any Warning or Danger event, appending a new record. Each payload includes: voltage, temperature, humidity, state string, fan boolean, buzzer boolean, `thermal_risk` boolean, and a server-side timestamp.

Cloud communication failure does not affect local actuator operation. Firebase calls are made after all actuator updates are applied, and any exception is silently caught. The firmware continues the polling loop regardless of HTTP response status.

G. Firmware Optimisation

Several design optimisations were applied to improve measurement reliability and reduce false activations.

ADC averaging. The SEN0565 analog output is sampled 32 times per reading cycle inside `readAnalogVoltage()`, and the mean value is used for state classification. This averaging suppresses ADC noise and sensor jitter that would otherwise cause spurious threshold crossings on individual samples.

Two-pass classification ordering. The state classification algorithm performs the voltage threshold check in the first pass and the BME280 thermal escalation check in the second pass. This ordering ensures that the computationally cheaper ADC read and voltage comparison execute before the I2C sensor read, keeping the critical path short when no escalation is required.

Gas level confirmation debounce. The `confirmGasLevel()` function requires five consecutive readings above a threshold before the state is promoted to Danger. This debounce prevents a single transient voltage spike — caused by sensor noise or a brief gas puff — from triggering full alarm activation, reducing nuisance alarms without increasing the worst-case detection latency beyond 10 seconds.

Adaptive baseline tracking. The baseline voltage follows ambient drift using a slow exponential moving average that updates only when the current reading is below the Warning threshold. This allows the system to adapt to gradual environmental changes such as temperature-induced sensor drift without incorporating leak events into the baseline, preserving threshold accuracy over time.

V. Web Dashboard

A. Technology and Architecture

The web dashboard is a single-page application implemented in plain HTML, CSS, and JavaScript with no build tools or frameworks. Chart.js is loaded from CDN for the trend charts. The Firebase JavaScript SDK (compat version 10.12.0) is loaded from CDN for real-time database access. The dashboard is hosted on Firebase Hosting at `leaksense-iot.web.app`. A Chrome service worker enables browser push notifications via the Web Push API, delivering alerts even when the dashboard tab is not open.

The JavaScript is organised into five files: `secrets.js` provides the local Firebase web configuration from an ignored file; `mock.js` provides simulated sensor data for development without hardware; `charts.js` encapsulates all Chart.js initialisation and update logic; `dashboard.js` contains the main state management and DOM manipulation; `firebase.js` contains the Firebase initialisation and real-time listeners.

B. Dashboard Features

Table III summarises the dashboard features and their technical implementation.

TABLE III
LeakSense web dashboard features and implementation.

Feature	Description	Implementation
Live gas voltage	Current voltage with Safe/Warning/Danger/Extreme label	Firebase `onValue()` WebSocket listener
Status badge & banner	Colour-coded state indicator updating in real time	CSS class toggling via JS on every reading
Device states panel	Fan, buzzer, and all three LED states shown live	DOM updates from Firebase payload fields
60-reading live chart	Voltage over time with threshold lines at 1.60, 1.90, 2.20 V	Chart.js line chart updated on each reading
24-hour history chart	Gas, temperature, and humidity over 24 h — dual Y-axis	`leaksense/history` query, Chart.js
24-hour reading table	Tabular log: voltage, temp, humidity, state	History path rendered to DOM table
Alert log	Timestamped log of all Warning, Danger, Extreme events	JS array rendered to DOM on each state change
Push notifications	Browser notifications on alert — tab may be closed	Chrome service worker + Web Push API
Thermal risk flag	Alert log flags combined gas-and-heat events	`thermal_risk` field from Firebase payload

C. Real-Time Update Mechanism

The dashboard uses the Firebase `onValue()` listener which establishes a persistent WebSocket connection to the Realtime Database. When the ESP32 pushes a new reading to `leaksense/latest`, Firebase pushes the update to all connected clients immediately. The dashboard receives the update and calls the `onNewReading()` function, which updates all UI elements in sequence. The entire update cycle from new data arrival to UI render completes in under 100 ms in testing.

The Safe dashboard state, shown in Fig. 10, represents normal ambient operation. The live voltage is below the 1.60 V warning threshold, so the interface presents a green status banner, keeps the fan and buzzer marked as inactive, and confirms that only the green LED is enabled. This view is the default monitoring state for day-to-day use.

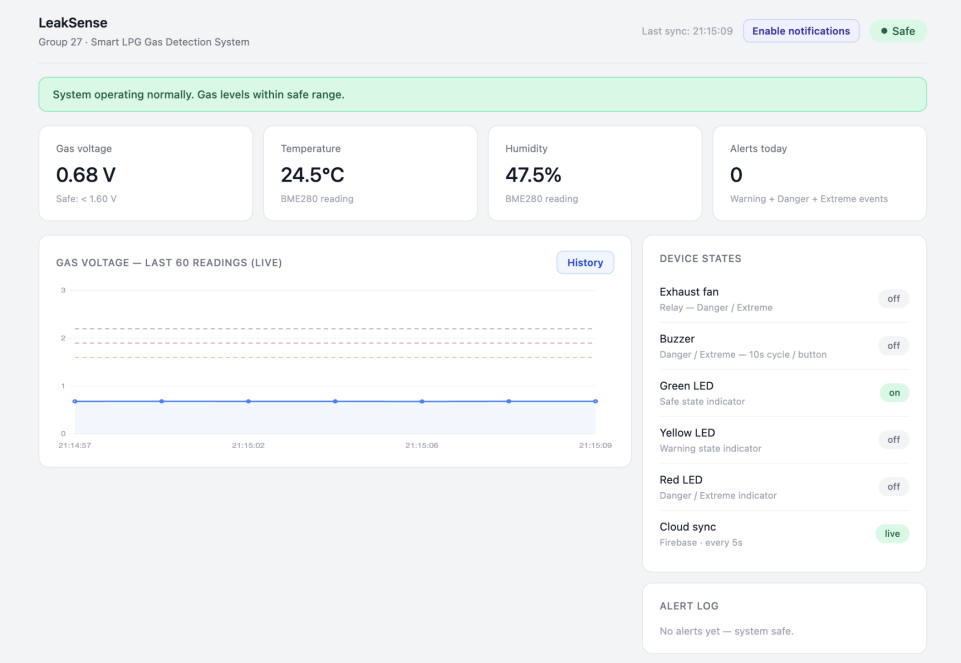

Fig. 10. LeakSense web dashboard — Safe state showing live gas voltage (0.68 V), system status, device states panel, and live trend chart.

When the gas voltage rises into the 1.60–1.89 V range, the dashboard changes to Warning as shown in Fig. 11. At this level, the system indicates elevated gas concentration but does not yet activate the fan or buzzer. The warning view is useful because it gives the user an early visual and remote indication before the system reaches the Danger threshold.

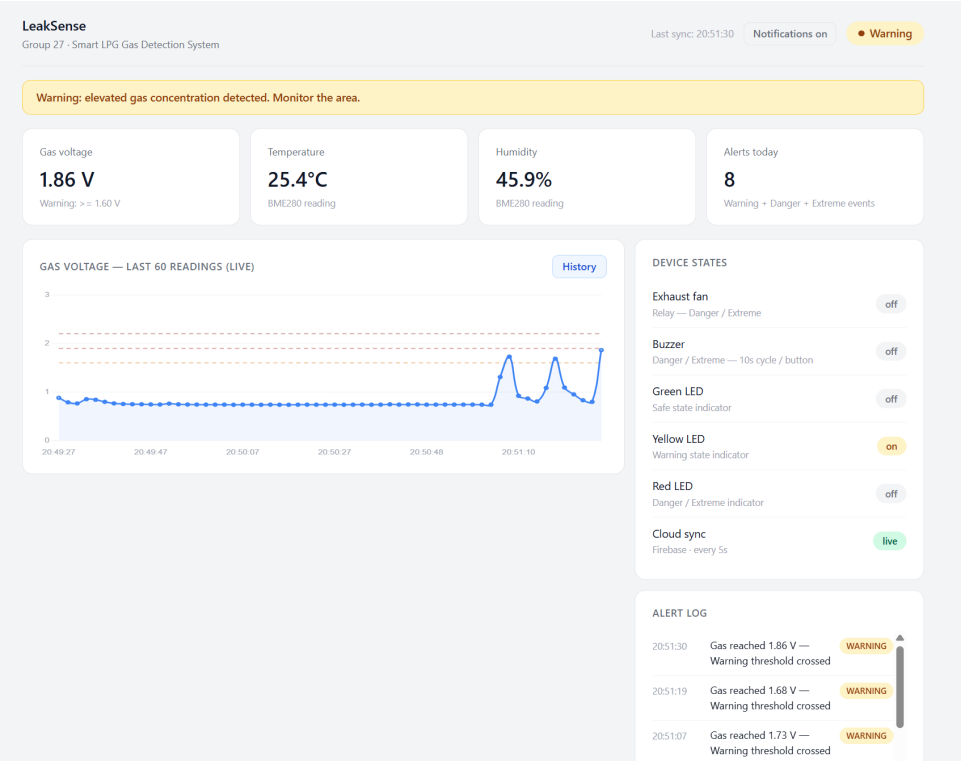

Fig. 11. LeakSense web dashboard — Warning state at 1.86 V with Chrome push notification visible.

Fig. 12 shows the Danger state, triggered when gas voltage reaches 1.90 V or above. In this state the dashboard confirms that the red LED is active, the relay-driven fan is on, and the buzzer alarm cycle has started. This matches the firmware behaviour

and verifies that the cloud dashboard is displaying the same safety state as the local hardware.

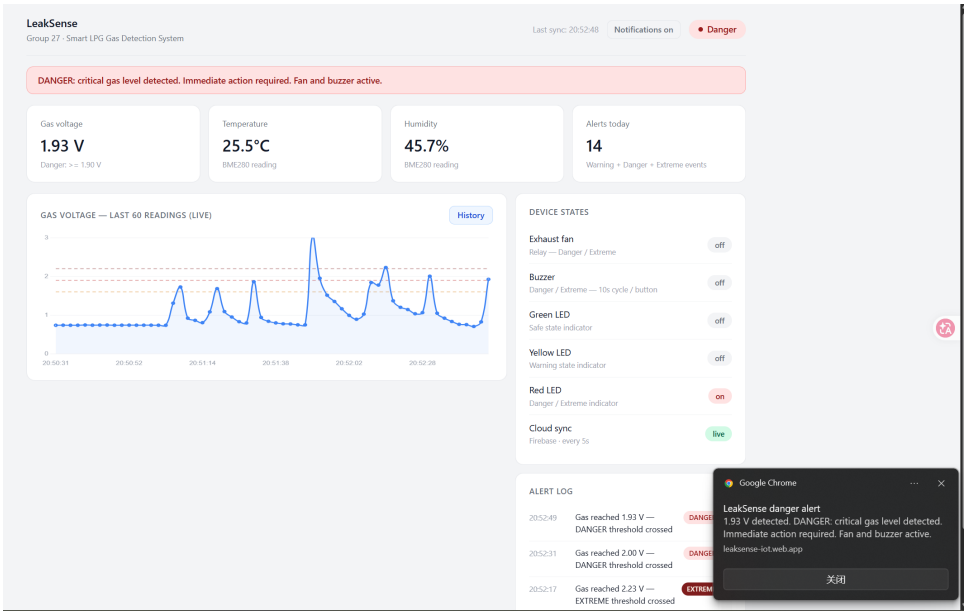

Fig. 12. LeakSense web dashboard — Danger state at 1.93 V with Chrome push notification visible.

The Extreme view in Fig. 13 is a dashboard-level escalation label for very high Danger readings above 2.20 V. The firmware still treats this condition as Danger, so the same local outputs remain active; however, the dashboard uses the Extreme label to make the remote alert more urgent and easier to distinguish from lower-level danger readings.

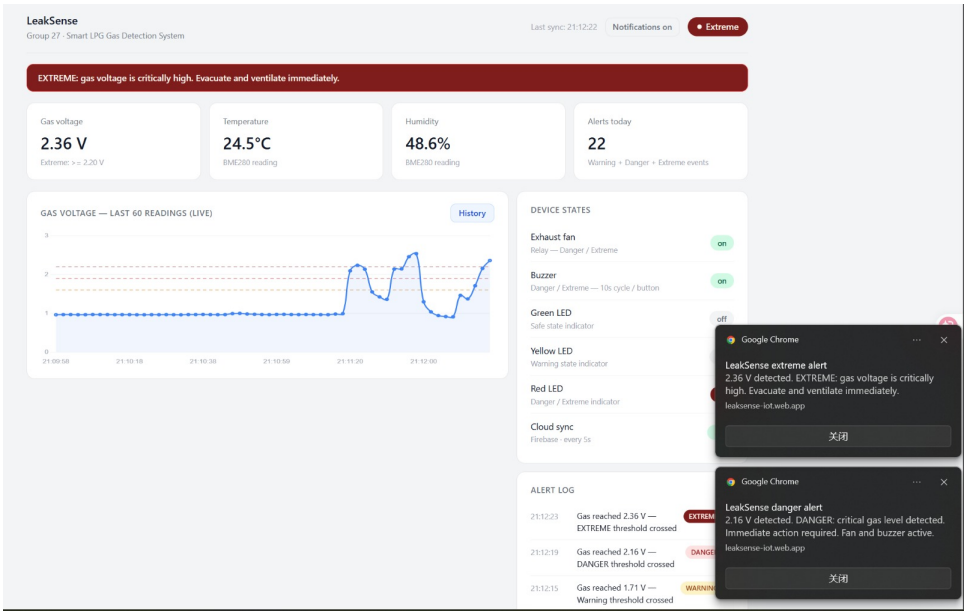

Fig. 13. LeakSense web dashboard — Extreme state at 2.36 V with Chrome push notification visible.

D. 24-Hour History View

The history view complements the live dashboard by showing how gas voltage, temperature, and humidity change over time. Rather than displaying only the latest Firebase value, it reads records from `leaksense/history` and plots the stored samples on a 24-hour chart. This helps identify recurring patterns, such as repeated short warning events, sensor warm-up drift, or environmental changes that coincide with gas readings.

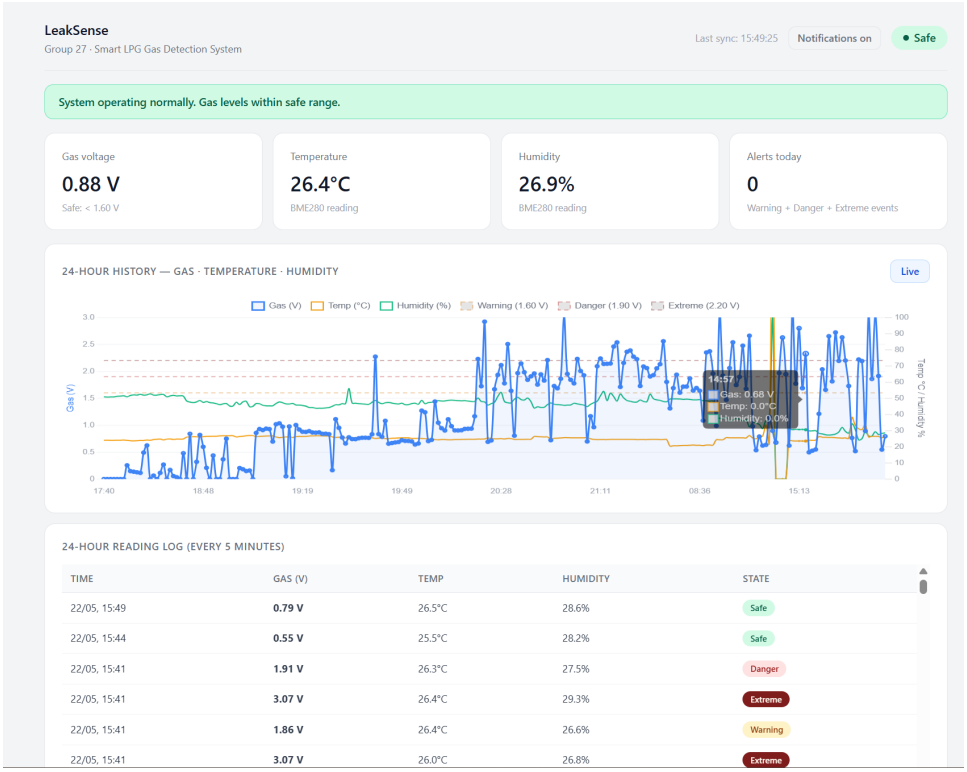

Fig. 14. LeakSense 24-hour history view showing stored sensor readings and state changes over time.

The table below the chart provides the same historical records in a readable log format, including timestamp, voltage, environmental values, and state. This is important for post-event analysis because a user can review when an alert occurred, how long elevated readings persisted, and whether the temperature trend supported the thermal escalation logic.

VI. Testing and Results

A. Testing Methodology

Testing was conducted in four phases on the UWA IoT laboratory bench. The SEN0565 sensor was allowed to warm up for a minimum of five minutes before each test session, with the ten-second baseline period observed on every boot, consistent with manufacturer warm-up guidance [6].

Phase 1 verified individual component communication: I2C addresses were confirmed via serial monitor scan, sensor readings were checked against expected ambient values, and actuator outputs were verified with a multimeter and direct visual inspection.

Phase 2 tested state machine behaviour using butane lighter gas held at measured distances from the sensor, recording the voltages at which state transitions occurred. The thermal escalation logic was tested separately by observing the temperature delta field across consecutive readings during deliberate ambient temperature changes.

Phase 3 tested cloud communication by monitoring the Firebase console during sensor operation and measuring the time between a reading being taken and data appearing in the database.

Phase 4 tested the full end-to-end system including dashboard responsiveness, push notification delivery, 24-hour history logging, and behaviour during Wi-Fi disconnection.

B. Test Results

Table IV summarises the results of the nine key test cases conducted.

TABLE IV
LEAKSENSE TEST CASE RESULTS.

#	Condition	Expected	Observed	Time	Result
01	Ambient air (baseline)	Safe · green LED on	Safe · 0.68 V · green LED on	<1 s	PASS
02	Lighter gas — moderate	Warning · yellow LED	Warning at 1.86 V · yellow LED on	∼2 s	PASS
03	Lighter gas — sustained	Danger · fan + buzzer	Danger at 1.93 V · fan + buzzer	∼2 s	PASS
04	Lighter gas — direct (stove)	Danger fw · Extreme display	2.36 V · Extreme on dashboard	∼2 s	PASS
05	Firebase latest push	Data within 2 s	Confirmed in console <2 s	<2 s	PASS
06	Dashboard live update	Update within 3 s	Dashboard updated <3 s	<3 s	PASS
07	Push notification	Chrome alert fires	Chrome notification received	∼3 s	PASS
08	Wi-Fi disconnected mid-Danger	Local actuators continue	Fan + buzzer unaffected	0 s	PASS
09	Button press during alarm	Silences current cycle	Buzzer off · restarts next cycle	<0.1 s	PASS

C. Results Analysis

All nine test cases passed. The system correctly classified gas voltage into all four display states and activated the appropriate actuators within the two-second polling interval. The two-second response time is determined by the sensor polling interval rather than processing latency — classification and actuator update execute in under one millisecond once the sensor reading is available.

Firebase push latency of under two seconds is within the target and is primarily determined by network round-trip time. Dashboard update latency of under three seconds reflects the combined Firebase push latency and the `onValue()` callback overhead. Push notification delivery at approximately three seconds reflects additional service worker processing time.

The Wi-Fi disconnection test confirmed the most important safety property of the design: local actuators operate entirely independently of cloud connectivity. When Wi-Fi was disabled during a Danger state, the fan and buzzer continued operating and the OLED continued updating, while Firebase push attempts were silently skipped. The button test confirmed that the current buzzer cycle is silenced immediately on press but the subsequent cycle restarts automatically when the gas remains in the Danger state.

The live demonstration using a gas stove (no ignition) confirmed real-world operation: voltage climbed from a Safe baseline of 0.68 V to 2.36 V within four seconds of the stove valve opening, triggering Danger and the Extreme dashboard label, activating the fan and buzzer, and firing a Chrome push notification before any team member present could detect the gas by smell.

VII. LIMITATIONS AND FUTURE WORK

A. Current Limitations

1) Empirical Voltage Thresholds: The most significant limitation is that the voltage thresholds (1.60 V for Warning, 1.90 V for Danger) are empirical calibration constants determined empirically during laboratory testing. The SEN0565 MEMS sensor is documented by DFRobot as a qualitative sensor only — it does not produce calibrated concentration measurements [5]. No manufacturer-published voltage-to-concentration mapping exists for this sensor. For a production safety system, formal calibration against certified LPG reference gas or a known percentage-LEL standard (OSHA specifies propane LEL at 2.1% by volume [7]) would be required. The system architecture accommodates recalibration: threshold constants are defined at the top of the firmware source and can be updated without structural changes.

2) Exhaust Fan Scale: The exhaust fan used in LeakSense is a small 12 V unit suitable for demonstrating relay-controlled ventilation but would not provide sufficient airflow to disperse LPG in a real room. A production system would require a mains-powered exhaust fan or a direct connection to an existing kitchen ventilation system.

3) No Physical Gas Shutoff Valve: The system activates ventilation and issues alerts but does not shut off the gas supply at the source. A solenoid valve integrated into the LPG supply line would provide a more complete safety response by eliminating the hazard rather than only mitigating it.

4) Firebase Security Configuration: The database currently operates with open read/write rules appropriate for a development configuration. A production deployment must implement Firebase Authentication and restrict write access to the ESP32 device credential only, with read access restricted to authenticated dashboard users.

5) Single Sensor Node and No Battery Backup: One gas sensor provides a single measurement point; a larger installation would benefit from multiple nodes. The system has no battery backup — a mains power failure disables all monitoring and local safety responses.

B. Future Work

Planned improvements include: (1) formal voltage-to-concentration calibration using certified LPG test gas to establish defensible thresholds based on percentage-LEL values; (2) a solenoid shutoff valve on the LPG supply line integrated with the existing relay driver circuit; (3) a UPS module or lithium battery backup for the ESP32; (4) production-grade Firebase security rules with device authentication; (5) a multi-node sensor mesh using ESP-NOW for larger residential spaces; (6) Firebase Cloud Functions for automatic history record pruning beyond 24 hours; and (7) dashboard email and SMS alert channels in addition to browser push notifications.

VIII. Project Cost

Table V gives the component cost estimate. Components were sourced from Core Electronics, Soldered, and Element14.

TABLE V
LeakSense component costs.

No.	Item	Description	Qty	Cost	Web Address
1	FireBeetle Board ESP32-E	Core controller with Wi-Fi and Bluetooth for IoT communication.	1	$18.80	https://core-electronics.com.au/firebeetle-board-esp32-e-arduino-compatible.html
2	SEN0565 MEMS Methane CH4 Gas Detection Sensor	Primary gas sensing input used for leakage detection.	1	$8.55	https://core-electronics.com.au/fermion-mems-methane-ch4-gas-detection-sensor-breakout-1-10000ppm.html
3	BME280 Atmospheric Sensor	Temperature, humidity, and pressure sensor used to reduce false alarms.	1	$23.50	https://core-electronics.com.au/piicodev-atmospheric-sensor-bme280.html
4	OLED Display SSD1306 (I2C)	0.96 in I2C SSD1306 screen for local status and data monitoring.	1	$7.20	https://core-electronics.com.au/white-i2c-oled-display-ssd1306.html
5	5 V Relay Module	Switches the fan load from ESP32 GPIO and provides electrical isolation.	1	$3.95	https://core-electronics.com.au/5v-single-channel-relay-module-10a.html
6	Delta 5 V DC Exhaust Fan 40 mm	5 V DC 40 mm 3150 RPM fan used for ventilation simulation.	1	$16.50	https://core-electronics.com.au/delta-electronics-frameless-fan-47192.html
7	Gravity Digital Buzzer Module	Gravity module used to provide audible alerts and system warnings.	1	$3.14	https://core-electronics.com.au/digital-buzzer-module.html
8	5 mm LED	Visual state indicator.	3	$3.14	https://au.element14.com/broadcom-limited/hlmp-3507/led-5mm-green-4-2mcd-569nm/dp/1003214
9	Resistors (270 Ohm)	Current limiting for LEDs; 600-pack covers all values.	1	$2.95	https://core-electronics.com.au/600-pack-of-1-4-watt-1-resistors-30-values-20-of-each.html
10	Breadboard 830 tie-pt	Prototyping platform for all connections.	1	$2.95	https://core-electronics.com.au/solderless-breadboard-830-tie-point-zy-102.html
11	Jumper Wires M-M and M-F	Breadboard interconnects; M-F wires are used for sensor modules and the OLED.	1	$3.59	https://core-electronics.com.au/solderless-breadboard-jumper-cable-wires-75-pieces.html
			Total	**$94.27**	

Competing commercial products provide context for this cost. The Aqara Smart Gas Detector (AUD $130) detects methane, not LPG. The Google Nest Protect (AUD $189) detects smoke and carbon monoxide, not LPG. The Honeywell BW Solo detects LPG but has no auto ventilation and costs approximately AUD $700. The Flowtech safety system includes a shutoff valve but starts at AUD $1,500 and is designed for commercial installations. LeakSense delivers LPG detection, relay-driven fan ventilation, cloud monitoring, 24-hour history, and browser push notifications for approximately AUD $94. The thermal escalation logic — gas plus temperature risk combined — is absent from every commercial alternative listed above.

IX. Conclusion

LeakSense demonstrates that an affordable, integrated residential LPG safety system can be built from commodity IoT components for approximately AUD $94 — roughly one-seventh the cost of the nearest commercial product with comparable functionality, and one-sixteenth the cost of systems offering certified shutoff integration. LeakSense integrates a MEMS analog gas sensor, a BME280 atmospheric sensor, relay-driven actuators, an ESP32 microcontroller, Firebase Realtime Database over HTTPS REST, and a browser-accessible dashboard with 24-hour history and push notifications into a coherent five-layer IoT architecture.

The four design goals defined in Section II were achieved and verified through nine test cases. Voltage-based state classification correctly identified Safe, Warning, Danger, and Extreme conditions at every tested threshold. The thermal escalation

logic correctly promoted Warning to Danger when a temperature rise of 5 °C or higher coincided with elevated gas readings, supporting the dual-sensor approach as a practical mechanism for distinguishing pure gas events from combined gas-and-heat events. Actuators responded within the two-second polling interval, cloud-to-dashboard latency remained under three seconds, and the Wi-Fi disconnection test confirmed the project's most important safety property: all local actuation continues independently of cloud connectivity.

Beyond the working system, the project contributes a reproducible reference architecture for low-cost residential safety IoT. The separation of time-critical safety logic on the microcontroller from non-critical telemetry and visualisation in the cloud is a pattern that generalises to other household hazard domains, including smoke, carbon monoxide, and water leak detection. The empirical voltage thresholds, small-scale fan, and absence of a physical shutoff valve are acknowledged limitations rather than design flaws; each has a defined remediation path in Section VII, and none require structural redesign of the existing architecture.

For practitioners, the principal takeaway is that the technical gap between affordable consumer detectors and certified industrial systems is smaller than current market pricing implies. With formal calibration against a certified LPG reference, a mains-rated exhaust fan, a solenoid shutoff valve, and production-grade Firebase security rules, the LeakSense architecture provides a credible foundation for a deployable open residential gas safety product.

X. How-To Guide

This guide describes how to assemble, configure, and run LeakSense from scratch.

A. Hardware Setup

1) Connect the SEN0565 gas sensor analog output to GPIO 39. Connect VCC to 3.3 V and GND to GND.
2) Connect the OLED SSD1306 to I2C Bus 0: SDA to GPIO 18, SCL to GPIO 23. Connect VCC to 3.3 V.
3) Connect the BME280 to I2C Bus 1: SDA to GPIO 22, SCL to GPIO 21. Connect VCC to 3.3 V.
4) Connect the HL-52S relay IN1 signal pin to GPIO 26. Connect the relay NO terminal in series with the fan's positive supply.
5) Connect the buzzer signal pin to GPIO 25.
6) Connect the green LED anode through a 270 Ω resistor to GPIO 17, cathode to GND.
7) Connect the yellow LED anode through a 270 Ω resistor to GPIO 16, cathode to GND.
8) Connect the red LED anode through a 270 Ω resistor to GPIO 4, cathode to GND.
9) Connect one terminal of the push button to GPIO 14, the other to GND. The firmware enables the internal pull-up resistor.
10) Power the ESP32 via USB-C. Verify I2C connections using the serial monitor I2C scan output before uploading firmware.

B. Firmware Installation

11) Install VS Code and the PlatformIO IDE extension.
12) Open the `firmware/` folder as a PlatformIO project.
13) Copy `firmware/blinking/src/secrets.h.example` to `firmware/blinking/src/secrets.h` and fill in: `WIFI_SSID`, `WIFI_PASSWORD`, `FIREBASE_HOST`, and `FIREBASE_AUTH`.
14) Ensure the following are listed in `platformio.ini` under `lib_deps`: Adafruit GFX Library, Adafruit SSD1306, Adafruit BME280 Library, and `mobizt/Firebase ESP32 Client`.
15) Click the PlatformIO Upload button to compile and flash the firmware.
16) Open the Serial Monitor at 115200 baud. Allow five minutes for sensor warm-up before trusting readings.

C. Firebase Setup

17) Go to firebase.google.com and create a project named `leaksense-iot`.
18) Enable Realtime Database in test mode.
19) In Project Settings → Service Accounts → Database Secrets, copy the database secret into the local firmware `secrets.h` file as `FIREBASE_AUTH`.
20) Copy the `databaseURL` from Project Settings and paste it (without `https://`) into the same `secrets.h` file as `FIREBASE_HOST`.
21) Set database rules to allow read and write for development. Update rules before any production deployment.

D. Dashboard Setup

22) Copy `dashboard/js/secrets.example.js` to `dashboard/js/secrets.js`. The local `secrets.js` file is ignored by Git and should not be committed.
23) Paste the Firebase web config object from Project Settings → General → Your apps into `dashboard/js/secrets.js`.

24) Open `dashboard/index.html` in a browser. The dashboard connects to Firebase automatically and displays live data.
25) To enable push notifications, open the dashboard in Chrome and click Enable Notifications. Accept the browser permission prompt. Notifications will fire for Warning, Danger, and Extreme events.
26) The 24-hour history view is accessible by clicking the History button on the live chart.

REFERENCES

[1] Frontier Economics, "Gas Vision 2050: Delivering a Clean Energy Future," Gas Energy Australia, Sep. 2020. [Online]. Available: https://www.gasenergyaus.au/get/1868/gas-vision-2050-frontier-economics-september.pdf. [Accessed: May 2026].
[2] Fire and Rescue NSW, "Annual Report 2024–25," Fire and Rescue NSW, Oct. 2025. [Online]. Available: https://www.fire.nsw.gov.au/__data/assets/pdf_file/0022/4936/annual_report_2024_25.pdf. [Accessed: May 2026].
[3] M. R. Islam *et al.*, "IoT-based smart gas leakage detection and safety system using wireless sensor networks," *IEEE Access*, vol. 8, pp. 201360–201371, 2020, doi: 10.1109/ACCESS.2020.3035486.
[4] R. Sarkar *et al.*, "Elevating Gas Safety Standards: ESP32-based Detection Systems with Blynk Integration," in *Proc. 2024 IEEE Region 10 Symposium (TENSYMP)*, 2024.
[5] DFRobot, "SEN0565 Fermion MEMS CH4 Gas Detection Sensor Wiki," DFRobot, 2024. [Online]. Available: https://wiki.dfrobot.com/sen0565. [Accessed: May 2026].
[6] DFRobot, "SEN0565 Raw Reading Example — warm-up guidance," DFRobot, 2024. [Online]. Available: https://wiki.dfrobot.com/sen0565/docs/18020. [Accessed: May 2026].
[7] OSHA, "LPG Chemical Data — Lower Explosive Limit," United States Department of Labor, 2024. [Online]. Available: https://www.osha.gov/chemicaldata/484. [Accessed: May 2026].
[8] L. Wu, L. Chen, and X. Hao, "Multi-sensor data fusion algorithm for early indoor fire warning based on BP neural network," *Information*, vol. 12, no. 2, p. 59, 2021.
[9] Espressif Systems, "ESP32 Technical Reference Manual," Espressif Systems, 2026. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf. [Accessed: May 2026].
[10] Bosch Sensortec, "BME280 Combined Humidity and Pressure Sensor — Data Sheet," Document BST-BME280-DS002, Rev. 1.6, Sep. 2018. [Online]. Available: https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf. [Accessed: May 2026].
[11] Google Firebase, "Firebase Realtime Database Documentation," Google, 2024. [Online]. Available: https://firebase.google.com/docs/database. [Accessed: May 2026].
[12] F. Saeed, A. Paul, A. Rehman, W. H. Hong, and H. Seo, "IoT-based intelligent modeling of smart home environment for fire prevention and safety," *J. Sensor Actuator Networks*, vol. 7, no. 1, p. 11, Mar. 2018, doi: 10.3390/jsan7010011. [Online]. Available: https://www.mdpi.com/2224-2708/7/1/11. [Accessed: May 2026].
[13] Core Electronics, "White I2C OLED Display (SSD1306)," Core Electronics, 2020. [Online]. Available: https://core-electronics.com.au/white-i2c-oled-display-ssd1306.html. [Accessed: May 2026].
[14] DFRobot, "Gravity: Digital Buzzer Module," DFRobot Wiki, 2024. [Online]. Available: https://wiki.dfrobot.com/dfr0032/. [Accessed: May 2026].
[15] Core Electronics, "5V Single Channel Relay Module 10A," Core Electronics, 2024. [Online]. Available: https://core-electronics.com.au/5v-single-channel-relay-module-10a.html. [Accessed: May 2026].
[16] Core Electronics, "Delta Electronics Frameless Fan," Core Electronics, 2024. [Online]. Available: https://core-electronics.com.au/delta-electronics-frameless-fan-47192.html. [Accessed: May 2026].
[17] Core Electronics, "Solderless Breadboard 830 Tie-Point ZY-102," Core Electronics, 2024. [Online]. Available: https://core-electronics.com.au/solderless-breadboard-830-tie-point-zy-102.html. [Accessed: May 2026].

APPENDIX A
CODE FUNCTION EXPLANATIONS

The code function explanations and source listings document the implementation used by LeakSense. The same source code is also available in the project GitHub repository: https://github.com/sahilppatel1807/CITS5506-LeakSense-IoT. For hardware assembly, firmware upload, Firebase setup, and dashboard operation instructions, see the How-To Guide.

A. Code Organisation

The LeakSense implementation is divided into embedded firmware and browser dashboard code. The embedded firmware is stored in `firmware/blinking/` and runs on the FireBeetle ESP32-E using PlatformIO and the Arduino framework. The dashboard is stored in `dashboard/` and runs as a plain HTML, CSS, and JavaScript application connected to Firebase Realtime Database. This split keeps time-critical safety logic on the ESP32 and presentation logic in the browser.

Table VI summarises the files included in Appendix B. The source listings are included directly from the repository so the report shows the same code used by LeakSense.

TABLE VI
LEAKSENSE SOURCE FILES INCLUDED IN APPENDIX B.

File	Role
firmware/blinking/src/main.cpp	ESP32 firmware: sensors, state machine, actuators, OLED display, Wi-Fi, Firebase REST upload.
firmware/blinking/platformio.ini	PlatformIO board, framework, monitor, and library dependency configuration.
firmware/blinking/src/secrets.h.example	Template for private Wi-Fi and Firebase credentials.
dashboard/index.html	Dashboard page structure and UI elements.
dashboard/css/style.css	Dashboard layout, state colours, cards, buttons, tables, and responsive styling.
dashboard/js/secrets.example.js	Template for the local Firebase web configuration file used by the dashboard.
dashboard/js/firebase.js	Firebase setup, database listeners, and data normalisation.
dashboard/js/dashboard.js	Main dashboard state handling, alert log, notifications, and DOM updates.
dashboard/js/charts.js	Chart.js live trend and 24-hour history charts.
dashboard/js/mock.js	Simulated readings for testing the dashboard without live hardware.
dashboard/service-worker.js	Browser notification service worker.

B. Firmware Functions Explained

The firmware is written with descriptive constants and small functions so that each hardware responsibility is visible in the code. Pin numbers, timing intervals, thresholds, active-low polarity, and calibration values are declared at the top of `main.cpp`. This makes the safety thresholds and GPIO assignments easy to audit and change.

The most important firmware functions are:

- `setup()` initialises GPIO, runs the LED self-test, configures ADC resolution, scans both I2C buses, initialises the OLED and BME280, and starts Wi-Fi connection in the background.
- `loop()` is the main cooperative scheduler. It services alarm controls and Wi-Fi continuously, then performs a complete sensor, classification, actuator, Firebase, and OLED update every two seconds.
- `readAnalogVoltage()` averages 32 ADC samples from the SEN0565 gas sensor, reducing noise before state classification.
- `updateGasBaseline()` tracks the clean-air baseline slowly when readings remain stable, but avoids learning elevated gas readings as normal.
- `evaluateGasLevel()` converts gas voltage into Safe, Warning, or Danger using the fixed thresholds.
- `confirmGasLevel()` prevents noisy single-sample danger spikes from triggering ordinary Danger, while allowing immediate Danger when the reading exceeds the Extreme voltage threshold.
- `hasThermalRisk()` checks whether the BME280 temperature has risen by at least 5 °C between readings or reached 55 °C absolute.
- `startDangerAlarm()`, `updateDangerAlarm()`, and `stopDangerAlarm()` control the buzzer and fan alarm cycle.
- `handleAlarmCancelButton()` handles the GPIO 14 push button and pauses detection temporarily after user acknowledgement.
- `firebasePayload()`, `uploadLatestToFirebase()`, and `uploadHistoryToFirebase()` build JSON records and send them to Firebase using HTTPS REST calls.
- `drawStatusScreen()` renders local OLED status including temperature, humidity, gas voltage, baseline, state, alarm, and fan status.

The firmware comments explain why timing, baselining, and upload behaviour are implemented in this way. The most important safety choice is that actuator updates occur before Firebase communication, so the fan, buzzer, and LEDs do not depend on Wi-Fi or cloud availability.

C. Dashboard Functions Explained

The dashboard JavaScript separates Firebase input, UI state handling, charts, and mock data. This makes it possible to test the browser interface without the ESP32 while keeping the same internal reading format.

The key dashboard functions are:

- `secrets.example.js` shows the required Firebase web configuration fields. The real local file is named `secrets.js` and is ignored by Git so project details are not committed.

- `normaliseReading()` in `firebase.js` converts Firebase records into one consistent object shape, handling missing fields, legacy names, booleans, numbers, timestamps, and state classification.
- `listenToSensorData()` subscribes to `leaksense/latest` so the dashboard updates in real time when the ESP32 uploads a new reading.
- `loadHistory()` and `listenToHistory()` read and stream `leaksense/history` records for the 24-hour history view.
- `onNewReading()` in `dashboard.js` is the main UI update function. It updates the state badge, banner, metrics, device state pills, live chart, alert log, and notifications.
- `sendAlertNotification()` creates browser notifications for Warning, Danger, and Extreme events.
- `setupDashboardViews()` switches between the live dashboard and 24-hour history views.
- `initChart()`, `updateChart()`, `initHistoryChart()`, and `addHistoryPoint()` in `charts.js` create and update the Chart.js visualisations.
- `getMockReading()` in `mock.js` produces simulated gas, temperature, humidity, actuator, and state values when hardware data is unavailable.

The HTML file defines the visible dashboard structure, the CSS file defines the state-specific visual design, and the JavaScript files apply the live Firebase data to those elements.

APPENDIX B
SOURCE CODE LISTINGS

A. *Firmware Source Code*

Listing 1. ESP32 firmware source code: firmware/blinking/src/main.cpp.

```cpp
#include <Arduino.h>
#include <Wire.h>
#include <WiFi.h>
#include <HTTPClient.h>
#include <WiFiClientSecure.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_BME280.h>
#include "secrets.h"

// LeakSense firmware:
// - Samples the SEN0565 gas sensor and BME280 environmental sensor.
// - Classifies gas risk into Safe, Warning, and Danger states.
// - Drives local outputs first, then uploads telemetry to Firebase if Wi-Fi is
        available.

// Hardware pin mapping and calibration constants are kept together so the
// wiring and threshold values can be audited easily from one place.
constexpr uint8_t kLedPin = D9;
constexpr uint8_t kGreenLedPin = 17;
constexpr uint8_t kRedLedPin = 4;
constexpr uint8_t kYellowLedPin = 16;
constexpr uint8_t kAlarmCancelButtonPin = 14;
constexpr uint8_t kOledSdaPin = 18;
constexpr uint8_t kOledSclPin = 23;
constexpr uint8_t kBmeSdaPin = 22;
constexpr uint8_t kBmeSclPin = 21;
constexpr uint8_t kAlarmOutputPin = 25;
constexpr uint8_t kFanRelayPin = 26;
constexpr int kSen0565AnalogPin = 39;
constexpr uint8_t kScreenWidth = 128;
constexpr uint8_t kScreenHeight = 64;
constexpr uint8_t kOledAddresses[] = {0x3C, 0x3D};
constexpr uint8_t kBmeAddresses[] = {0x76, 0x77};
constexpr uint8_t kGasSamplesPerRead = 32;
constexpr unsigned long kBlinkIntervalMs = 500;
constexpr unsigned long kRedBlinkIntervalMs = 75;
constexpr unsigned long kSensorReadIntervalMs = 2000;
constexpr unsigned long kFirebaseHistoryIntervalMs = 300000;
constexpr unsigned long kWifiConnectTimeoutMs = 15000;
constexpr unsigned long kWifiRetryIntervalMs = 30000;
constexpr unsigned long kGasBaselineDurationMs = 10000;
constexpr unsigned long kLedSelfTestIntervalMs = 500;
constexpr unsigned long kDangerAlarmDurationMs = 10000;
constexpr unsigned long kDetectionPauseDurationMs = 10000;
constexpr float kSen0565WarningVoltageThresholdV = 1.60f;
constexpr float kSen0565DangerVoltageThresholdV = 1.90f;
constexpr float kSen0565ExtremeVoltageThresholdV = 2.20f;
constexpr float kThermalRiskTempRiseC = 5.0f;
constexpr float kThermalRiskAbsoluteTempC = 55.0f;
constexpr uint8_t kGasLevelConfirmReads = 5;
constexpr bool kAlarmOutputActiveLow = true;
constexpr bool kFanRelayActiveLow = true;
constexpr float kGasBaselineFollowAlpha = 0.02f;
constexpr float kGasBaselineSettleDeltaV = 0.05f;
constexpr float kGasBaselineSettleRatio = 1.03f;

// Firmware-level gas state. The dashboard later labels very high Danger
// readings as "Extreme", but the ESP32 treats them as Danger for actuation.
enum class GasLevel {
  Safe,
  Warning,
  Danger,
};

// Runtime state for the SEN0565 sensor, including live readings and clean-air
// baseline values used for drift tracking and rough ppm estimation.
struct GasSensorState {
  const char* name;
  int analogPin;
  float warningVoltageThresholdV;
  float dangerVoltageThresholdV;
  float extremeVoltageThresholdV;
  float baselineVoltage;
  bool baselineReady;
  int raw;
  float voltage;
  GasLevel level;
  bool extreme;
  bool detected;
};

TwoWire gOledWire = TwoWire(0);
TwoWire gBmeWire = TwoWire(1);
Adafruit_SSD1306 display(kScreenWidth, kScreenHeight, &gOledWire, -1);
Adafruit_BME280 gBme;

bool gDisplayReady = false;
bool gBmeReady = false;
uint8_t gDisplayAddress = 0;
uint8_t gBmeAddress = 0;

bool gWiFiConnected = false;
bool gWiFiConnecting = false;
unsigned long gWiFiConnectStartMs = 0;
unsigned long gLastWiFiAttemptMs = 0;

bool gLedOn = false;

bool gGasDetected = false;
bool gDangerAlarmActive = false;
bool gDetectionPaused = false;
bool gRedBlinkOn = false;
bool gAlarmCancelButtonLatched = false;
GasLevel gGasLevel = GasLevel::Safe;
GasLevel gLastGasLevel = GasLevel::Safe;
GasLevel gPendingGasLevel = GasLevel::Safe;
uint8_t gPendingGasLevelCount = 0;
unsigned long gStartMs = 0;
unsigned long gLastBlinkMs = 0;
unsigned long gLastRedBlinkMs = 0;
unsigned long gLastSensorReadMs = 0;
unsigned long gLastFirebaseHistoryMs = 0;
unsigned long gDangerAlarmStartMs = 0;
unsigned long gDetectionPauseStartMs = 0;
float gLastTemperatureC = NAN;
GasSensorState gSen0565Sensor = {"SEN0565",
                                 kSen0565AnalogPin,
                                 kSen0565WarningVoltageThresholdV,
                                 kSen0565DangerVoltageThresholdV,
                                 kSen0565ExtremeVoltageThresholdV,
                                 0.0f,
                                 false,
                                 0,
                                 0.0f,
                                 GasLevel::Safe,
                                 false,
                                 false};

// Converts the internal enum into the string stored in Serial output,
// Firebase records, OLED status, and dashboard state handling.
const char* gasLevelName(GasLevel level) {
  switch (level) {
    case GasLevel::Danger:
      return "DANGER";
    case GasLevel::Warning:
      return "WARNING";
```

```
    case GasLevel::Safe:
    default:
      return "SAFE";
  }
}

// Relay and buzzer modules are active-low in this prototype, so these helpers
// hide the polarity and let the rest of the firmware use true/false safely.
void setAlarmOutput(bool enabled) {
  const uint8_t activeState = kAlarmOutputActiveLow ? LOW : HIGH;
  const uint8_t inactiveState = kAlarmOutputActiveLow ? HIGH : LOW;
  digitalWrite(kAlarmOutputPin, enabled ? activeState : inactiveState);
}

void setFanRelay(bool enabled) {
  const uint8_t activeState = kFanRelayActiveLow ? LOW : HIGH;
  const uint8_t inactiveState = kFanRelayActiveLow ? HIGH : LOW;
  digitalWrite(kFanRelayPin, enabled ? activeState : inactiveState);
}

void setStatusLeds(bool greenOn, bool redOn, bool yellowOn);
void handleAlarmCancelButton(unsigned long now);
void updateDangerAlarm(unsigned long now);
void updateStatusLeds(unsigned long now);
void serviceAlarmControls(unsigned long now);
void startWiFiConnection(unsigned long now);
void serviceWiFi(unsigned long now);

// Starts one local Danger response cycle. This happens before any Firebase
// upload so local safety action does not depend on the network.
void startDangerAlarm(unsigned long now) {
  gDangerAlarmActive = true;
  gDangerAlarmStartMs = now;
  gRedBlinkOn = true;
  gLastRedBlinkMs = now;
  setAlarmOutput(true);
  setFanRelay(true);
}

// Clears the current alarm cycle and returns the visible outputs to Safe.
void stopDangerAlarm() {
  gDangerAlarmActive = false;
  gGasDetected = false;
  gGasLevel = GasLevel::Safe;
  gPendingGasLevel = GasLevel::Safe;
  gPendingGasLevelCount = 0;
  gRedBlinkOn = false;
  setAlarmOutput(false);
  setFanRelay(false);
  setStatusLeds(true, false, false);
}

// User acknowledgement silences the alarm temporarily, but monitoring resumes
// automatically after the pause window.
void startDetectionPause(unsigned long now) {
  stopDangerAlarm();
  gDetectionPaused = true;
  gDetectionPauseStartMs = now;
}

void updateDetectionPause(unsigned long now) {
  if (!gDetectionPaused) {
    return;
  }

  if (now - gDetectionPauseStartMs >= kDetectionPauseDurationMs) {
    gDetectionPaused = false;
    gLastSensorReadMs = 0;
    Serial.println("Detection pause ended; monitoring resumed");
    return;
  }
}

void updateDangerAlarm(unsigned long now) {
  if (!gDangerAlarmActive) {
    setAlarmOutput(false);
    setFanRelay(false);
    return;
  }

  if (now - gDangerAlarmStartMs >= kDangerAlarmDurationMs) {
    stopDangerAlarm();
    return;
  }

  setAlarmOutput(true);
  setFanRelay(true);
}

void setStatusLeds(bool greenOn, bool redOn, bool yellowOn) {
  digitalWrite(kGreenLedPin, greenOn ? HIGH : LOW);
  digitalWrite(kRedLedPin, redOn ? HIGH : LOW);
  digitalWrite(kYellowLedPin, yellowOn ? HIGH : LOW);
}

void runLedSelfTest() {
  setStatusLeds(false, false, false);
  delay(100);
  Serial.println("LED self-test: green -> yellow -> red");

  setStatusLeds(true, false, false);
  delay(kLedSelfTestIntervalMs);
  setStatusLeds(false, false, true);
  delay(kLedSelfTestIntervalMs);
  setStatusLeds(false, true, false);
  delay(kLedSelfTestIntervalMs);
  setStatusLeds(false, false, false);
}

// Wi-Fi is managed non-blockingly so sensor sampling and actuator control keep
// running even when the router is unavailable.
void startWiFiConnection(unsigned long now) {
  if (WiFi.status() == WL_CONNECTED || gWiFiConnecting) {
    return;
  }

  Serial.printf("Starting WiFi connection to SSID='%s'...\r\n", WIFI_SSID);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  gWiFiConnecting = true;
  gWiFiConnectStartMs = now;
  gLastWiFiAttemptMs = now;
}

void serviceWiFi(unsigned long now) {
  const wl_status_t status = WiFi.status();

  if (status == WL_CONNECTED) {
    if (!gWiFiConnected) {
      Serial.printf("WiFi connected, IP=%s\r\n", WiFi.localIP().toString().
          c_str());
    }
    gWiFiConnected = true;
    gWiFiConnecting = false;
    return;
  }

  if (gWiFiConnected) {
    Serial.println("WiFi disconnected. Local monitoring continues; Firebase
        upload paused.");
  }
  gWiFiConnected = false;

  if (gWiFiConnecting) {
    if (now - gWiFiConnectStartMs >= kWifiConnectTimeoutMs) {
      Serial.println("WiFi connection timed out. Local monitoring continues;
          will retry later.");
      WiFi.disconnect(false);
      gWiFiConnecting = false;
    }
    return;
  }

  if (gLastWiFiAttemptMs == 0 || now - gLastWiFiAttemptMs >=
      kWifiRetryIntervalMs) {
    startWiFiConnection(now);
  }
}

float gasRatio(const GasSensorState& sensor);
float gasNormalizedRatio(const GasSensorState& sensor);
float gasDeltaMagnitude(const GasSensorState& sensor);

// The SEN0565 output is qualitative, so this ppm estimate is only for display.
// Safety decisions are made from voltage thresholds, not from this estimate.
float estimatePpm(const GasSensorState& sensor) {
  if (!sensor.baselineReady || sensor.baselineVoltage <= 0.0f) {
    return 0.0f;
  }

  const float ratio = gasNormalizedRatio(sensor);
  if (ratio <= 1.0f) {
    return 0.0f;
  }
  if (ratio <= 2.2f) {
    return (ratio - 1.0f) * 250.0f;
  }
  return 300.0f + (ratio - 2.2f) * 500.0f;
}

// Firebase does not accept NaN or Infinity, so invalid sensor values are sent
// as JSON null instead of malformed numeric text.
String jsonFloat(float value, unsigned int decimalPlaces) {
  if (isnan(value) || isinf(value)) {
    return "null";
  }
  return String(value, decimalPlaces);
}

// Builds the JSON payload shared by latest and history uploads.
String firebasePayload(float ppm,
                       float rawPpm,
                       float gasVoltage,
                       float temperatureC,
                       float humidityPct,
                       const char* state,
                       bool fan,
                       bool buzzer,
                       bool thermalRisk) {
  String payload = "{";
  payload += "\"ppm_compensated\":" + jsonFloat(ppm, 1) + ",";
  payload += "\"ppm_raw\":" + jsonFloat(rawPpm, 1) + ",";
  payload += "\"voltage\":" + jsonFloat(gasVoltage, 3) + ",";
  payload += "\"temperature\":" + jsonFloat(temperatureC, 1) + ",";
  payload += "\"humidity\":" + jsonFloat(humidityPct, 1) + ",";
  payload += "\"state\":\"" + String(state) + "\",";
  payload += "\"fan\":" + String(fan ? "true" : "false") + ",";
  payload += "\"buzzer\":" + String(buzzer ? "true" : "false") + ",";
  payload += "\"thermal_risk\":" + String(thermalRisk ? "true" : "false") + ","
      ;
  payload += "\"timestamp\":{\".sv\":\"timestamp\"}";
  payload += "}";
  return payload;
}

// Sends one reading to Firebase. PUT updates leaksense/latest, while POST
// appends a new record under leaksense/history.
bool uploadReadingToFirebase(const char* node,
                             bool append,
                             float ppmCompensated,
                             float ppmRaw,
                             float gasVoltage,
```

```
                              float temperatureC,
                              float humidityPct,
                              const char* state,
                              bool fan,
                              bool buzzer,
                              bool thermalRisk) {
  if (WiFi.status() != WL_CONNECTED) {
    return false;
  }

  WiFiClientSecure client;
  client.setInsecure();
  HTTPClient http;
  const String url = String("https://") + FIREBASE_HOST + "/leaksense/" + node
      + ".json?auth=" + FIREBASE_AUTH;

  if (!http.begin(client, url)) {
    Serial.println("Firebase: begin failed");
    return false;
  }

  http.addHeader("Content-Type", "application/json");
  const String payload = firebasePayload(ppmCompensated,
                                         ppmRaw,
                                         gasVoltage,
                                         temperatureC,
                                         humidityPct,
                                         state,
                                         fan,
                                         buzzer,
                                         thermalRisk);
  const int httpCode = append ? http.POST(payload) : http.PUT(payload);

  if (httpCode <= 0) {
    Serial.printf("Firebase: request failed (%d)\r\n", httpCode);
    http.end();
    return false;
  }

  if (httpCode >= 200 && httpCode < 300) {
    Serial.printf("Firebase: uploaded %s reading, code=%d\r\n", node, httpCode)
        ;
    http.end();
    return true;
  }

  Serial.printf("Firebase: upload error %d -> %s\r\n", httpCode, http.
      errorToString(httpCode).c_str());
  http.end();
  return false;
}

// Latest is the live value used by the dashboard's real-time listener.
bool uploadLatestToFirebase(float ppmCompensated,
                            float ppmRaw,
                            float gasVoltage,
                            float temperatureC,
                            float humidityPct,
                            const char* state,
                            bool fan,
                            bool buzzer,
                            bool thermalRisk) {
  return uploadReadingToFirebase("latest",
                                 false,
                                 ppmCompensated,
                                 ppmRaw,
                                 gasVoltage,
                                 temperatureC,
                                 humidityPct,
                                 state,
                                 fan,
                                 buzzer,
                                 thermalRisk);
}

// History is sampled less frequently, plus every alert, to support 24-hour
      review.
bool uploadHistoryToFirebase(float ppmCompensated,
                             float ppmRaw,
                             float gasVoltage,
                             float temperatureC,
                             float humidityPct,
                             const char* state,
                             bool fan,
                             bool buzzer,
                             bool thermalRisk) {
  return uploadReadingToFirebase("history",
                                 true,
                                 ppmCompensated,
                                 ppmRaw,
                                 gasVoltage,
                                 temperatureC,
                                 humidityPct,
                                 state,
                                 fan,
                                 buzzer,
                                 thermalRisk);
}

// Applies LED states for Safe, Warning, Danger, and the temporary pause mode.
void updateStatusLeds(unsigned long now) {
  if (gDetectionPaused) {
    updateDetectionPause(now);
    return;
  }

  if (gDangerAlarmActive) {
    if (now - gLastRedBlinkMs >= kRedBlinkIntervalMs) {
      gLastRedBlinkMs = now;
      gRedBlinkOn = !gRedBlinkOn;
    }
    setStatusLeds(false, gRedBlinkOn, false);
    return;
  }

  switch (gGasLevel) {
    case GasLevel::Danger:
      setStatusLeds(false, true, false);
      break;
    case GasLevel::Warning:
      setStatusLeds(false, false, true);
      break;
    case GasLevel::Safe:
    default:
      setStatusLeds(true, false, false);
      break;
  }
}

// Runs the high-priority controls on every loop pass and during ADC averaging.
void serviceAlarmControls(unsigned long now) {
  handleAlarmCancelButton(now);
  updateDangerAlarm(now);
  updateStatusLeds(now);
}

// GPIO14 is wired with INPUT_PULLUP, so LOW means the button is pressed.
void handleAlarmCancelButton(unsigned long now) {
  const bool pressed = digitalRead(kAlarmCancelButtonPin) == LOW;
  if (!pressed) {
    gAlarmCancelButtonLatched = false;
    return;
  }

  if (gAlarmCancelButtonLatched) {
    return;
  }

  gAlarmCancelButtonLatched = true;
  Serial.println("GPIO14 button pressed: alarm stopped; gas detection paused
       for 10 seconds");
  startDetectionPause(now);
}

// Danger is confirmed across several readings to avoid noisy single-sample
// spikes, while Extreme voltage still escalates immediately.
GasLevel confirmGasLevel(GasLevel newLevel, bool extremeDanger) {
  if (newLevel == GasLevel::Safe) {
    gPendingGasLevel = GasLevel::Safe;
    gPendingGasLevelCount = 0;
    return GasLevel::Safe;
  }

  if (newLevel == GasLevel::Warning) {
    gPendingGasLevel = GasLevel::Warning;
    gPendingGasLevelCount = 0;
    return GasLevel::Warning;
  }

  if (extremeDanger) {
    gPendingGasLevel = GasLevel::Danger;
    gPendingGasLevelCount = 0;
    return GasLevel::Danger;
  }

  if (gPendingGasLevel != GasLevel::Danger) {
    gPendingGasLevel = GasLevel::Danger;
    gPendingGasLevelCount = 1;
  } else if (gPendingGasLevelCount < kGasLevelConfirmReads) {
    ++gPendingGasLevelCount;
  }
  if (gPendingGasLevelCount >= kGasLevelConfirmReads) {
    gPendingGasLevelCount = 0;
    return GasLevel::Danger;
  }

  return GasLevel::Warning;
}

// Prints detected I2C addresses to help debug OLED and BME280 wiring.
void scanI2CBus(TwoWire& bus, const char* label) {
  Serial.printf("Scanning %s I2C bus...\r\n", label);
  for (uint8_t address = 1; address < 127; ++address) {
    bus.beginTransmission(address);
    if (bus.endTransmission() == 0) {
      Serial.printf("%s bus device found at 0x%02X\r\n", label, address);
    }
  }
}

bool initDisplay() {
  for (uint8_t address : kOledAddresses) {
    if (display.begin(SSD1306_SWITCHCAPVCC, address)) {
      gDisplayAddress = address;
      return true;
    }
  }
  return false;
}

bool initBme() {
  for (uint8_t address : kBmeAddresses) {
    if (gBme.begin(address, &gBmeWire)) {
      gBmeAddress = address;
      return true;
    }
  }
  return false;
}

// Averages 32 ADC samples to smooth sensor noise. Alarm controls are serviced
// during the short sample delay so the cancel button remains responsive.
```

```
float readAnalogVoltage(int analogPin) {
  uint32_t total = 0;
  for (uint8_t i = 0; i < kGasSamplesPerRead; ++i) {
    total += analogRead(analogPin);
    serviceAlarmControls(millis());
    delay(2);
  }

  const float raw = total / static_cast<float>(kGasSamplesPerRead);
  return raw * 3.3f / 4095.0f;
}

// Learns a clean-air baseline during startup, then follows slow drift only
// while readings remain close to normal.
void updateGasBaseline(GasSensorState& sensor) {
  if (sensor.baselineVoltage <= 0.0f) {
    sensor.baselineVoltage = sensor.voltage;
    return;
  }

  if (!sensor.baselineReady) {
    sensor.baselineVoltage = (sensor.baselineVoltage * 0.85f) + (sensor.voltage
        * 0.15f);
    if (millis() - gStartMs >= kGasBaselineDurationMs) {
      sensor.baselineReady = true;
      Serial.printf("%s baseline ready: %.3f V\r\n", sensor.name, sensor.
       baselineVoltage);
    }
    return;
  }

  if (gasDeltaMagnitude(sensor) < kGasBaselineSettleDeltaV &&
      gasNormalizedRatio(sensor) < kGasBaselineSettleRatio) {
    sensor.baselineVoltage =
        (sensor.baselineVoltage * (1.0f - kGasBaselineFollowAlpha)) +
        (sensor.voltage * kGasBaselineFollowAlpha);
  }
}

// Ratio helpers compare the live gas voltage against the learned baseline.
float gasRatio(const GasSensorState& sensor) {
  return sensor.baselineVoltage > 0.01f ? sensor.voltage / sensor.
      baselineVoltage : 0.0f;
}

float gasNormalizedRatio(const GasSensorState& sensor) {
  const float ratio = gasRatio(sensor);
  if (ratio <= 0.0f) {
    return 1.0f;
  }
  return ratio >= 1.0f ? ratio : (1.0f / ratio);
}

float gasDeltaMagnitude(const GasSensorState& sensor) {
  return fabsf(sensor.voltage - sensor.baselineVoltage);
}

// Converts voltage thresholds into the local safety state.
GasLevel evaluateGasLevel(const GasSensorState& sensor) {
  if (sensor.voltage >= sensor.extremeVoltageThresholdV) {
    return GasLevel::Danger;
  }
  if (sensor.voltage >= sensor.dangerVoltageThresholdV) {
    return GasLevel::Danger;
  }
  if (sensor.voltage >= sensor.warningVoltageThresholdV) {
    return GasLevel::Warning;
  }
  return GasLevel::Safe;
}

// Adds the second-sensor escalation rule: gas risk plus abnormal temperature
// rise or high absolute temperature becomes Danger.
bool hasThermalRisk(float temperatureC) {
  if (!isfinite(temperatureC)) {
    return false;
  }

  const bool highTemperature = temperatureC >= kThermalRiskAbsoluteTempC;
  const bool rapidRise = isfinite(gLastTemperatureC) &&
                         (temperatureC - gLastTemperatureC) >=
      kThermalRiskTempRiseC;

  return highTemperature || rapidRise;
}

// Reads and updates all SEN0565 fields used by classification, Firebase, and
      OLED.
void sampleGasSensor(GasSensorState& sensor) {
  sensor.raw = analogRead(sensor.analogPin);
  sensor.voltage = readAnalogVoltage(sensor.analogPin);
  updateGasBaseline(sensor);
  sensor.extreme = sensor.baselineReady &&
                   sensor.voltage >= sensor.extremeVoltageThresholdV;
  sensor.level = evaluateGasLevel(sensor);
  sensor.detected = sensor.level == GasLevel::Danger;
}

// Serial output is used during bench testing to verify threshold transitions.
void printGasSensor(const GasSensorState& sensor) {
  if (sensor.baselineReady) {
    const float signedDelta = sensor.voltage - sensor.baselineVoltage;
    Serial.printf("%s -> raw=%d, V=%.3f, baseline=%.3f, delta=%+.3f, absDelta
       =%.3f, ratio=%.2f, state=%s, extreme=%s\r\n",
                  sensor.name,
                  sensor.raw,
                  sensor.voltage,
                  sensor.baselineVoltage,
                  signedDelta,
                  gasDeltaMagnitude(sensor),
                  gasNormalizedRatio(sensor),
                  gasLevelName(sensor.level),
                  sensor.extreme ? "YES" : "no");
  } else {
    Serial.printf("%s baselining -> raw=%d, V=%.3f\r\n",
                  sensor.name,
                  sensor.raw,
                  sensor.voltage);
  }
}

// Local OLED display mirrors the main safety state so the hardware remains
// understandable even without the web dashboard.
void drawStatusScreen(float temperatureC,
                      float humidityPct,
                      float pressureHpa,
                      const GasSensorState& sensor,
                      GasLevel confirmedLevel,
                      bool gasDetected,
                      bool fanOn) {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);
  display.println("LeakSense SEN0565");

  if (gBmeReady) {
    display.printf("T:%.1fC H:%.0f%% P:%.0f\r\n", temperatureC, humidityPct,
       pressureHpa);

  } else {
    display.println("BME280 not found");

  }

  display.printf("Raw:%4d V:%.2f\r\n", sensor.raw, sensor.voltage);
  if (sensor.baselineReady) {
    display.printf("Base:%.2f R:%.2f\r\n", sensor.baselineVoltage,
       gasNormalizedRatio(sensor));
    display.printf("State:%s\r\n", gasLevelName(confirmedLevel));
    display.printf("Alarm:%s Fan:%s\r\n", gasDetected ? "ON" : "off", fanOn ? "
       ON" : "off");


  } else {
    const unsigned long elapsedMs = millis() - gStartMs;
    const unsigned long cappedElapsedMs =
        elapsedMs > kGasBaselineDurationMs ? kGasBaselineDurationMs : elapsedMs
       ;
    const unsigned long remainingSec = (kGasBaselineDurationMs -
       cappedElapsedMs) / 1000;
    display.println("Baselining SEN0565");

    display.printf("Time left: %lus\r\n", remainingSec);
  }

  display.display();
}

// Initialises GPIO, sensors, display, ADC settings, Wi-Fi, and safe output
      states.
void setup() {
  Serial.begin(115200);
  pinMode(kLedPin, OUTPUT);
  pinMode(kGreenLedPin, OUTPUT);
  pinMode(kRedLedPin, OUTPUT);
  pinMode(kYellowLedPin, OUTPUT);
  pinMode(kAlarmOutputPin, OUTPUT);
  pinMode(kFanRelayPin, OUTPUT);
  pinMode(kAlarmCancelButtonPin, INPUT_PULLUP);
  gAlarmCancelButtonLatched = false;
  setAlarmOutput(false);
  setFanRelay(false);
  runLedSelfTest();
  updateStatusLeds(millis());

  analogReadResolution(12);
  analogSetPinAttenuation(kSen0565AnalogPin, ADC_11db);
  gStartMs = millis();
  delay(200);

  Serial.printf("OLED test starting (SDA=%u, SCL=%u)\r\n", kOledSdaPin,
      kOledSclPin);
  gOledWire.begin(kOledSdaPin, kOledSclPin);
  scanI2CBus(gOledWire, "OLED");

  gDisplayReady = initDisplay();
  if (gDisplayReady) {
    Serial.printf("OLED initialized at 0x%02X\r\n", gDisplayAddress);
  } else {
    Serial.println("OLED init failed. Check VCC/GND/SDA/SCL and address.");
  }

  Serial.printf("BME280 test starting (SDA=%u, SCL=%u)\r\n", kBmeSdaPin,
      kBmeSclPin);
  gBmeWire.begin(kBmeSdaPin, kBmeSclPin);
  scanI2CBus(gBmeWire, "BME280");

  gBmeReady = initBme();
  if (gBmeReady) {
    Serial.printf("BME280 initialized at 0x%02X\r\n", gBmeAddress);
  } else {
    Serial.println("BME280 init failed. Check VCC/GND/SDA/SCL and ADR/CS pins."
       );
  }

  Serial.printf("%s test starting (A=GPIO%d)\r\n", gSen0565Sensor.name,
      gSen0565Sensor.analogPin);
  Serial.println("SEN0565 alarm contribution: enabled");
  Serial.println("Keep SEN0565 in clean air for the first 10 seconds.");
  Serial.println("Warm SEN0565 for at least 5 minutes before trusting its
      thresholds.");
```

```
  startWiFiConnection(millis());
  Serial.println("WiFi will connect in background. Local monitoring is
      available offline.");
}

// Main cooperative scheduler. Local controls run continuously; sensor sampling
    ,
// state classification, Firebase upload, and OLED refresh run every two
    seconds.
void loop() {
  const unsigned long now = millis();
  serviceAlarmControls(now);
  serviceWiFi(now);

  if (now - gLastBlinkMs >= kBlinkIntervalMs) {
    gLastBlinkMs = now;
    gLedOn = !gLedOn;
    digitalWrite(kLedPin, gLedOn ? HIGH : LOW);
  }

  if (!gDetectionPaused && now - gLastSensorReadMs >= kSensorReadIntervalMs) {
    gLastSensorReadMs = now;

    float temperatureC = NAN;
    float humidityPct = NAN;
    float pressureHpa = NAN;
    sampleGasSensor(gSen0565Sensor);
    if (gDetectionPaused) {
      return;
    }
    if (gBmeReady) {
      temperatureC = gBme.readTemperature();
      humidityPct = gBme.readHumidity();
      pressureHpa = gBme.readPressure() / 100.0f;

      Serial.printf("BME280 0x%02X -> T=%.2f C, H=%.2f %%, P=%.2f hPa\r\n",
                    gBmeAddress, temperatureC, humidityPct, pressureHpa);
    } else {
      Serial.println("BME280 not ready");
    }

    gLastGasLevel = gGasLevel;
    gGasLevel = confirmGasLevel(gSen0565Sensor.level, gSen0565Sensor.extreme);
    const bool thermalRisk = hasThermalRisk(temperatureC);
    if (thermalRisk) {
      gGasLevel = GasLevel::Danger;
      Serial.println("Thermal risk escalation: abnormal temperature rise/high
       temperature");
    }
    gGasDetected = gGasLevel == GasLevel::Danger;
    if (gGasDetected && gLastGasLevel != GasLevel::Danger) {
      startDangerAlarm(now);
    }
    updateDangerAlarm(now);
    updateStatusLeds(now);
    if (isfinite(temperatureC)) {
      gLastTemperatureC = temperatureC;
    }

    printGasSensor(gSen0565Sensor);
    Serial.printf("Alarm source -> SEN0565 raw=%s, confirmed=%s, alarm=%s, fan
      =%s\r\n",
                  gasLevelName(gSen0565Sensor.level),
                  gasLevelName(gGasLevel),
                  gDangerAlarmActive ? "ON" : "off",
                  gDangerAlarmActive ? "ON" : "off");

    // Upload latest every cycle for live dashboard display. Append history
      only
    // every five minutes or immediately when a Warning/Danger event occurs.
    const float ppmCompensated = estimatePpm(gSen0565Sensor);
    const float ppmRaw = gSen0565Sensor.raw * 0.25f;
    const char* stateName = gasLevelName(gGasLevel);
    const bool fanOn = gDangerAlarmActive;
    const bool buzzerOn = gDangerAlarmActive;
    const bool shouldUploadAlertHistory =
        gGasLevel == GasLevel::Warning || gGasLevel == GasLevel::Danger;

    if (WiFi.status() == WL_CONNECTED) {
      uploadLatestToFirebase(ppmCompensated,
                             ppmRaw,
                             gSen0565Sensor.voltage,
                             temperatureC,
                             humidityPct,
                             stateName,
                             fanOn,
                             buzzerOn,
                             thermalRisk);

      if (shouldUploadAlertHistory ||
          gLastFirebaseHistoryMs == 0 ||
          now - gLastFirebaseHistoryMs >= kFirebaseHistoryIntervalMs) {
        if (uploadHistoryToFirebase(ppmCompensated,
                                    ppmRaw,
                                    gSen0565Sensor.voltage,
                                    temperatureC,
                                    humidityPct,
                                    stateName,
                                    fanOn,
                                    buzzerOn,
                                    thermalRisk)) {
          gLastFirebaseHistoryMs = now;
        }
      }
    }

    if (gDisplayReady) {
      drawStatusScreen(temperatureC,
                       humidityPct,
                       pressureHpa,
                       gSen0565Sensor,
                       gGasLevel,
                       gDangerAlarmActive,
                       gDangerAlarmActive);
    }
  }
}
```

B. PlatformIO Configuration

Listing 2. PlatformIO configuration: firmware/blinking/platformio.ini.

```
; PlatformIO build configuration for the FireBeetle ESP32-E firmware.
; These settings select the target board, Arduino framework, Serial monitor
; speed, and the display/environment sensor libraries used by main.cpp.
[env:firebeetle2_esp32e]
platform = espressif32
board = dfrobot_firebeetle2_esp32e
framework = arduino
monitor_speed = 115200

; Libraries required for the OLED display and BME280 atmospheric sensor.
lib_deps =
  adafruit/Adafruit GFX Library @ ^1.12.1
  adafruit/Adafruit SSD1306 @ ^2.5.15
  adafruit/Adafruit BME280 Library @ ^2.3.0

; Project: LeakSense - Smart LPG Gas Leakage Detection and Automatic Safety
      System
```

C. Credential Template

Listing 3. Credential template: firmware/blinking/src/secrets.h.example.

```
// LeakSense - Smart LPG Gas Leakage Detection and Automatic Safety System
// Copy this file to secrets.h and replace the placeholder values below.
// secrets.h is ignored by Git so Wi-Fi and Firebase credentials stay private.
#pragma once

// Wi-Fi credentials used by the ESP32 for Firebase uploads.
#define WIFI_SSID "YOUR_WIFI_NAME"
#define WIFI_PASSWORD "YOUR_WIFI_PASSWORD"

// Firebase Realtime Database host and database secret used by REST requests.
#define FIREBASE_HOST "YOUR_PROJECT-default-rtdb.firebaseio.com"
#define FIREBASE_AUTH "YOUR_REALTIME_DATABASE_SECRET"
```

D. Dashboard HTML

Listing 4. Dashboard HTML: dashboard/index.html.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>LeakSense -- LPG Gas Detection Dashboard</title>

  <!-- Chart.js -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.1/dist/chart.umd.min.
      js"></script>

  <!-- Firebase -->
  <script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-app-compat.
      js"></script>
  <script src="https://www.gstatic.com/firebasejs/10.12.0/firebase-database-
      compat.js"></script>

  <link rel="stylesheet" href="css/style.css" />
</head>
<body>

<div class="app">

  <!-- -- Top bar ------------------------------------------------------------
      -->
  <header class="topbar">
    <div class="topbar__left">
      <span class="topbar__title">LeakSense</span>
      <span class="topbar__sub">Group 27 · Smart LPG Gas Detection System</span
      >
    </div>
    <div class="topbar__right">
      <span class="topbar__sync" id="lastSync">Last sync: --</span>
      <button class="notification-btn" type="button" id="enableNotificationsBtn
      " hidden>
        Enable notifications
      </button>
      <span class="status-badge safe" id="statusBadge">Safe</span>
    </div>
  </header>

  <!-- -- State notification bar ---------------------------------------------
      -->
  <div class="state-bar safe" id="stateBar">
    System operating normally. Gas levels within safe range.
  </div>

  <!-- -- Metric cards -------------------------------------------------------
      -->
  <section class="metrics">
    <div class="card metric-card">
      <p class="metric-card__label">Gas voltage</p>
      <p class="metric-card__value" id="ppmVal">-- V</p>
      <p class="metric-card__sub" id="ppmSub">Sensor output</p>
    </div>
    <div class="card metric-card">
      <p class="metric-card__label">Temperature</p>
```

```
      <p class="metric-card__value" id="tempVal">--°C</p>
      <p class="metric-card__sub">BME280 reading</p>
    </div>
    <div class="card metric-card">
      <p class="metric-card__label">Humidity</p>
      <p class="metric-card__value" id="humVal">--%</p>
      <p class="metric-card__sub">BME280 reading</p>
    </div>
    <div class="card metric-card">
      <p class="metric-card__label">Alerts today</p>
      <p class="metric-card__value" id="alertCount">0</p>
      <p class="metric-card__sub">Warning + Danger + Extreme events</p>
    </div>
  </section>

  <!-- -- Live main grid ------------------------------------------------------
       -->
  <div class="main-grid view-panel" id="liveView">

    <!-- Live trend chart -->
    <div class="card chart-card">
      <div class="card__header">
        <p class="card__title">Gas voltage -- last 60 readings (live)</p>
        <button class="view-toggle-btn" type="button" id="showHistoryBtn">
       History</button>
      </div>
      <canvas id="trendChart"></canvas>
    </div>

    <!-- Device states + alert log -->
    <div class="right-col">

      <!-- Device states -->
      <div class="card">
        <p class="card__title">Device states</p>
        <div class="device-list">

          <div class="device-row">
            <div>
              <p class="device-row__name">Exhaust fan</p>
              <p class="device-row__sub">Relay -- Danger / Extreme</p>
            </div>
            <span class="pill" id="fanState">off</span>
          </div>

          <div class="device-row">
            <div>
              <p class="device-row__name">Buzzer</p>
              <p class="device-row__sub">Danger / Extreme -- 10s cycle / button
      </p>
            </div>
            <span class="pill" id="buzzerState">off</span>
          </div>

          <div class="device-row">
            <div>
              <p class="device-row__name">Green LED</p>
              <p class="device-row__sub">Safe state indicator</p>
            </div>
            <span class="pill pill--on" id="greenLed">on</span>
          </div>

          <div class="device-row">
            <div>
              <p class="device-row__name">Yellow LED</p>
              <p class="device-row__sub">Warning state indicator</p>
            </div>
            <span class="pill" id="yellowLed">off</span>
          </div>

          <div class="device-row">
            <div>
              <p class="device-row__name">Red LED</p>
              <p class="device-row__sub">Danger / Extreme indicator</p>
            </div>
            <span class="pill" id="redLed">off</span>
          </div>

          <div class="device-row device-row--last">
            <div>
              <p class="device-row__name">Cloud sync</p>
              <p class="device-row__sub">Firebase · every 2s</p>
            </div>
            <span class="pill pill--on" id="cloudSync">live</span>
          </div>

        </div>
      </div>

      <!-- Alert log -->
      <div class="card alert-card">
        <p class="card__title">Alert log</p>
        <div class="alert-log" id="alertLog">
          <p class="alert-log__empty">No alerts yet -- system safe.</p>
        </div>
      </div>

    </div>
  </div><!-- /.main-grid -->

  <!-- -- 24-hour history -----------------------------------------------------
       -->
  <div class="history-section view-panel" id="historyView">

    <!-- 24hr chart -->
    <div class="card history-chart-card">
      <div class="card__header">
        <p class="card__title">24-hour history -- Gas · Temperature · Humidity
       </p>
        <button class="view-toggle-btn" type="button" id="showLiveBtn">Live</
       button>
      </div>
      <canvas id="historyChart"></canvas>
    </div>

    <!-- 24hr data table -->
    <div class="card">
      <p class="card__title">24-hour reading log (every 5 minutes)</p>
      <div class="history-table-wrap">
        <p class="history-empty" id="historyEmpty">Loading history...</p>
        <table class="history-table">
          <thead>
            <tr>
              <th>Time</th>
              <th>Gas (V)</th>
              <th>Temp</th>
              <th>Humidity</th>
              <th>State</th>
            </tr>
          </thead>
          <tbody id="historyTableBody"></tbody>
        </table>
      </div>
    </div>

  </div><!-- /.history-section -->

</div><!-- /.app -->

<!-- JS -- order matters -->
<!-- mock.js is loaded first so the dashboard can fall back to simulated data
     when Firebase credentials are not configured. -->
<script src="js/mock.js"></script>
<!-- charts.js exposes chart initialisation/update functions used by dashboard.
       js. -->
<script src="js/charts.js"></script>
<!-- secrets.js is local-only and defines window.LEAKSENSE_FIREBASE_CONFIG. -->
<script src="js/secrets.js"></script>
<!-- firebase.js listens to Realtime Database and normalises incoming readings.
       -->
<script src="js/firebase.js"></script>
<!-- dashboard.js wires UI events and updates the page from each reading. -->
<script src="js/dashboard.js"></script>

</body>
</html>
```

E. Dashboard CSS

Listing 5. Dashboard CSS: dashboard/css/style.css.

```
/* -- Reset & base ---------------------------------------------------------
      */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
  background: #f3f4f6;
  color: #111827;
  font-size: 14px;
  line-height: 1.5;
}

/* -- Layout ---------------------------------------------------------------
      */
.app {
  max-width: 1100px;
  margin: 0 auto;
  padding: 1.25rem;
}

/* -- Top bar --------------------------------------------------------------
      */
.topbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  margin-bottom: 1rem;
  padding-bottom: 1rem;
  border-bottom: 1px solid #e5e7eb;
}
.topbar__left  { display: flex; flex-direction: column; gap: 2px; }
.topbar__right { display: flex; align-items: center; gap: 12px; }
.topbar__title { font-size: 16px; font-weight: 600; color: #111827; }
.topbar__sub   { font-size: 12px; color: #6b7280; }
.topbar__sync  { font-size: 12px; color: #9ca3af; }

.notification-btn {
  /* Hidden until dashboard.js checks browser notification support. */
  border: 1px solid #c7d2fe;
  border-radius: 8px;
  background: #eef2ff;
  color: #3730a3;
  cursor: pointer;
  font: inherit;
  font-size: 12px;
  font-weight: 600;
  line-height: 1;
  padding: 7px 10px;
  transition: background 0.15s, border-color 0.15s, color 0.15s;
  white-space: nowrap;
}
.notification-btn:hover {
  background: #e0e7ff;
  border-color: #a5b4fc;
}
.notification-btn:focus-visible {
  outline: 3px solid rgba(99, 102, 241, 0.25);
  outline-offset: 2px;
}
```

```
.notification-btn:disabled {
  background: #f3f4f6;
  border-color: #e5e7eb;
  color: #6b7280;
  cursor: default;
}

/* -- Status badge ------------------------------------------------------------ */
.status-badge {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 4px 14px;
  border-radius: 20px;
  font-size: 13px;
  font-weight: 500;
  transition: background 0.3s, color 0.3s;
}
.status-badge::before {
  /* Small status dot inherits the badge colour for Safe/Warning/Danger. */
  content: '';
  display: inline-block;
  width: 7px;
  height: 7px;
  border-radius: 50%;
  background: currentColor;
}
.status-badge.safe    { background: #d1fae5; color: #065f46; }
.status-badge.warning { background: #fef3c7; color: #92400e; }
.status-badge.danger  { background: #fee2e2; color: #991b1b; }
.status-badge.extreme { background: #7f1d1d; color: #fff7ed; }

/* -- State notification bar -------------------------------------------------- */
.state-bar {
  padding: 10px 16px;
  border-radius: 10px;
  font-size: 13px;
  font-weight: 500;
  margin-bottom: 1rem;
  border: 1px solid transparent;
  transition: background 0.3s, color 0.3s;
}
.state-bar.safe    { background: #d1fae5; color: #065f46; border-color: #6ee7b7; }
.state-bar.warning { background: #fef3c7; color: #92400e; border-color: #fcd34d; }
.state-bar.danger  { background: #fee2e2; color: #991b1b; border-color: #fca5a5; }
.state-bar.extreme { background: #7f1d1d; color: #fff7ed; border-color: #ef4444; }

/* -- Card -------------------------------------------------------------------- */
.card {
  background: #ffffff;
  border: 1px solid #e5e7eb;
  border-radius: 12px;
  padding: 1.1rem 1.25rem;
}
.card__title {
  font-size: 12px;
  font-weight: 500;
  color: #6b7280;
  margin-bottom: 0.75rem;
  text-transform: uppercase;
  letter-spacing: 0.04em;
}
.card__header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  margin-bottom: 0.75rem;
}
.card__header .card__title { margin-bottom: 0; }

.view-panel[hidden] { display: none; }

.view-toggle-btn {
  border: 1px solid #bfdbfe;
  border-radius: 8px;
  background: #eff6ff;
  color: #1d4ed8;
  cursor: pointer;
  font: inherit;
  font-size: 12px;
  font-weight: 600;
  line-height: 1;
  padding: 7px 12px;
  transition: background 0.15s, border-color 0.15s, color 0.15s;
  white-space: nowrap;
}
.view-toggle-btn:hover {
  background: #dbeafe;
  border-color: #93c5fd;
  color: #1e40af;
}
.view-toggle-btn:focus-visible {
  outline: 3px solid rgba(59, 130, 246, 0.25);
  outline-offset: 2px;
}

/* -- Metric cards ------------------------------------------------------------ */
.metrics {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 12px;
  margin-bottom: 1rem;
}
.metric-card__label { font-size: 12px; color: #6b7280; margin-bottom: 4px; }
.metric-card__value { font-size: 24px; font-weight: 500; color: #111827; }
.metric-card__sub   { font-size: 11px; color: #9ca3af; margin-top: 3px; }

/* -- Main grid --------------------------------------------------------------- */
.main-grid {
  display: grid;
  grid-template-columns: 1fr 340px;
  gap: 12px;
  align-items: start;
  margin-bottom: 1rem;
}

/* -- Chart card -------------------------------------------------------------- */
.chart-card canvas {
  /* Fixed height prevents the live chart from resizing as points are added. */
  width: 100% !important;
  height: 200px !important;
}

/* -- Right column ------------------------------------------------------------ */
.right-col {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

/* -- Device list ------------------------------------------------------------- */
.device-list { display: flex; flex-direction: column; }

.device-row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 9px 0;
  border-bottom: 1px solid #f3f4f6;
}
.device-row--last { border-bottom: none; }
.device-row__name { font-size: 13px; color: #111827; }
.device-row__sub  { font-size: 11px; color: #9ca3af; margin-top: 1px; }

/* -- Pills ------------------------------------------------------------------- */
.pill {
  display: inline-block;
  padding: 3px 11px;
  border-radius: 20px;
  font-size: 11px;
  font-weight: 500;
  background: #f3f4f6;
  color: #6b7280;
  transition: background 0.2s, color 0.2s;
}
.pill--on      { background: #d1fae5; color: #065f46; }
.pill--warning { background: #fef3c7; color: #92400e; }
.pill--danger  { background: #fee2e2; color: #991b1b; }
.pill--extreme { background: #7f1d1d; color: #fff7ed; }

/* -- Alert log --------------------------------------------------------------- */
.alert-card  { flex: 1; }
.alert-log   { max-height: 220px; overflow-y: auto; }
.alert-log__empty { font-size: 12px; color: #9ca3af; }

.alert-row {
  display: flex;
  align-items: flex-start;
  gap: 8px;
  padding: 8px 0;
  border-bottom: 1px solid #f3f4f6;
}
.alert-row:last-child { border-bottom: none; }
.alert-row__time  { font-size: 11px; color: #9ca3af; white-space: nowrap; min-width: 60px; margin-top: 1px; }
.alert-row__msg   { font-size: 12px; color: #111827; flex: 1; }

.alert-pill {
  font-size: 10px;
  font-weight: 500;
  padding: 2px 8px;
  border-radius: 10px;
  white-space: nowrap;
}
.alert-pill--warning { background: #fef3c7; color: #92400e; }
.alert-pill--danger  { background: #fee2e2; color: #991b1b; }
.alert-pill--extreme { background: #7f1d1d; color: #fff7ed; }

/* -- 24hr history section ---------------------------------------------------- */
.history-section {
  display: flex;
  flex-direction: column;
  gap: 12px;
}

.history-chart-card canvas {
  width: 100% !important;
  height: 240px !important;
}

/* -- History table ----------------------------------------------------------- */
.history-table-wrap {
  overflow-x: auto;
  max-height: 320px;
  overflow-y: auto;
```

```
}

.history-table {
  width: 100%;
  border-collapse: collapse;
  font-size: 12px;
}

.history-table thead th {
  /* Sticky header keeps column names visible while scrolling 24-hour history. */
  position: sticky;
  top: 0;
  background: #f9fafb;
  color: #6b7280;
  font-weight: 500;
  text-align: left;
  padding: 8px 10px;
  border-bottom: 1px solid #e5e7eb;
  white-space: nowrap;
  text-transform: uppercase;
  font-size: 11px;
  letter-spacing: 0.03em;
}

.history-table tbody tr {
  border-bottom: 1px solid #f3f4f6;
  transition: background 0.15s;
}
.history-table tbody tr:last-child { border-bottom: none; }
.history-table tbody tr:hover { background: #f9fafb; }
.history-table tbody td {
  padding: 8px 10px;
  color: #374151;
  white-space: nowrap;
}

.hist-badge {
  display: inline-block;
  padding: 2px 9px;
  border-radius: 10px;
  font-size: 10px;
  font-weight: 500;
}
.hist-badge--safe    { background: #d1fae5; color: #065f46; }
.hist-badge--warning { background: #fef3c7; color: #92400e; }
.hist-badge--danger  { background: #fee2e2; color: #991b1b; }
.hist-badge--extreme { background: #7f1d1d; color: #fff7ed; }

.history-empty {
  font-size: 12px;
  color: #9ca3af;
  text-align: center;
  padding: 1.5rem 0;
}

/* -- Responsive ------------------------------------------------------------ */
@media (max-width: 768px) {
  /* Tablet/mobile layout stacks the chart above device and alert panels. */
  .topbar {
    align-items: flex-start;
    gap: 12px;
  }
  .topbar__right {
    flex-wrap: wrap;
    justify-content: flex-end;
  }
  .metrics   { grid-template-columns: repeat(2, 1fr); }
  .main-grid { grid-template-columns: 1fr; }
}
@media (max-width: 480px) {
  .topbar {
    flex-direction: column;
  }
  .topbar__right {
    justify-content: flex-start;
    width: 100%;
  }
  .metrics { grid-template-columns: 1fr; }
  .card__header {
    align-items: flex-start;
    flex-direction: column;
  }
}
```

F. Dashboard JavaScript

Listing 6. Dashboard Firebase credential template: dashboard/js/secrets.example.js.

```
/**
 * secrets.example.js
 * Copy this file to secrets.js and replace the placeholder values with the
 * Firebase web app config from Firebase Project Settings.
 *
 * secrets.js is intentionally ignored by Git so local Firebase project details
 * are not committed or printed in the report source listings.
 */
window.LEAKSENSE_FIREBASE_CONFIG = {
  apiKey: "YOUR_FIREBASE_WEB_API_KEY",
  authDomain: "YOUR_PROJECT.firebaseapp.com",
  databaseURL: "https://YOUR_PROJECT-default-rtdb.firebaseio.com",
  projectId: "YOUR_PROJECT",
  storageBucket: "YOUR_PROJECT.firebasestorage.app",
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
  appId: "YOUR_FIREBASE_APP_ID",
};
```

Listing 7. Firebase JavaScript: dashboard/js/firebase.js.

```
/**
* firebase.js
* Firebase Realtime Database connection.
* - leaksense/latest  → live reading (every 2s from ESP32)
* - leaksense/history → one entry every 5 minutes, kept for 24 hours
*/
const firebaseConfig = window.LEAKSENSE_FIREBASE_CONFIG;
const hasFirebaseConfig = firebaseConfig && Object.values(firebaseConfig).every(value =>
  typeof value === 'string' && value.trim() !== '' && !value.includes('YOUR_')
);

if (hasFirebaseConfig) {
  firebase.initializeApp(firebaseConfig);
} else {
  console.warn('Firebase config missing. Copy js/secrets.example.js to js/secrets.js and fill it in.');
}

const db = hasFirebaseConfig ? firebase.database() : null;

// -- Helpers ---------------------------------------------------------------

// Accepts multiple possible field names so older Firebase records and firmware
// payload changes still render correctly in the dashboard.
function numberFrom(...values) {
  const found = values.find(v => v !== undefined && v !== null && v !== '');
  const n = Number(found);
  return Number.isFinite(n) ? n : 0;
}

// Firebase may store booleans as real booleans or strings depending on source.
function booleanFrom(value) {
  if (typeof value === 'boolean') return value;
  if (typeof value === 'string')  return value.toLowerCase() === 'true' || value.toLowerCase() === 'on';
  return Boolean(value);
}

// Converts a raw Firebase object into the one consistent reading shape used by
// dashboard.js and charts.js.
function normaliseReading(data) {
  const ppm   = numberFrom(data.ppm_compensated, data.ppm, data.gas_ppm, data.gas, data.value);
  const suppliedVoltage = numberFrom(data.voltage, data.gas_voltage, data.voltage_v, data.sensor_voltage);
  const voltage = suppliedVoltage > 0 ? suppliedVoltage : gasVoltageFromPpm(ppm);
  const state = mostSevereState(data.state, getStateFromVoltage(voltage));
  const alarmActive = state === 'danger' || state === 'extreme';

  return {
    ppm_compensated: Math.round(ppm),
    ppm_raw:         Math.round(numberFrom(data.ppm_raw, data.raw_ppm, ppm)),
    voltage,
    temperature:     numberFrom(data.temperature, data.temp),
    humidity:        numberFrom(data.humidity, data.hum),
    state,
    thermal_risk: data.thermal_risk === undefined ? booleanFrom(data.thermalRisk) : booleanFrom(data.thermal_risk),
    fan:    data.fan    === undefined ? alarmActive : booleanFrom(data.fan),
    buzzer: data.buzzer === undefined ? alarmActive : booleanFrom(data.buzzer),
    timestamp: numberFrom(data.timestamp, data.time, Date.now()),
  };
}

// Firebase history is stored as keyed objects. This helper turns the snapshot
// into a sorted array so charts and tables display in time order.
function snapshotToReadings(snapshot) {
  const value = snapshot.val();
  if (!value || typeof value !== 'object' || Array.isArray(value)) return [];

  return Object.values(value)
    .filter(item => item && typeof item === 'object')
    .map(normaliseReading)
    .sort((a, b) => a.timestamp - b.timestamp);
}

// -- 24-hour history -------------------------------------------------------

const TWENTY_FOUR_HOURS_MS = 24 * 60 * 60 * 1000;

/**
* Loads all history entries from the last 24 hours.
* Called once on page load to populate the history chart and table.
* @param {function} callback - receives array of normalised reading objects
*/
function loadHistory(callback) {
  if (!db) {
    callback([]);
    return;
  }

  const cutoff = Date.now() - TWENTY_FOUR_HOURS_MS;

  db.ref('leaksense/history')
    .orderByChild('timestamp')
    .startAt(cutoff)
    .once('value')
    .then(snapshot => {
      const readings = snapshotToReadings(snapshot);
      callback(readings);
    })
    .catch(err => {
      console.warn('Could not load 24hr history:', err);
      callback([]);
    });
```

```
}

/**
* Listens for new history entries in real time.
* Only fires for entries newer than page load time.
* @param {function} callback - receives a single normalised reading
*/
function listenToHistory(callback) {
  if (!db) return false;

  const since = Date.now() - TWENTY_FOUR_HOURS_MS;

  db.ref('leaksense/history')
    .orderByChild('timestamp')
    .startAt(since)
    .on('child_added', snapshot => {
      const data = snapshot.val();
      if (!data) return;
      callback(normaliseReading(data));
    });

  return true;
}

// -- Live latest reading ------------------------------------------------------

/**
* Subscribes to leaksense/latest for real-time dashboard updates.
* @param {function} callback - onNewReading from dashboard.js
*/
function listenToSensorData(callback) {
  if (!db) return false;

  db.ref('leaksense/latest').on('value', snapshot => {
    const data = snapshot.val();
    if (!data) return;
    callback(normaliseReading(data));
  });

  return true;
}
```

Listing 8. Main dashboard JavaScript: dashboard/js/dashboard.js.

```
/**
* dashboard.js
* Main dashboard logic.
* Handles live readings, alert log, 24hr history table, and state machine.
*/

// -- Dashboard display thresholds ---------------------------------------------
const VOLTAGE_THRESHOLDS = { warning: 1.60, danger: 1.90, extreme: 2.20 };
const PPM_THRESHOLDS = { warning: 300, danger: 500, extreme: 800 };

// -- State --------------------------------------------------------------------
let alertCount = 0;
let prevState  = 'safe';
const alertLog = [];

// 24-hour history table data
const historyRows = [];
const MAX_HISTORY_ROWS = 288; // 24hr at 1 reading per 5 min

// -- Helpers ------------------------------------------------------------------

// Converts gas ppm into the same display state logic used by voltage readings.
function getStateFromPpm(ppm) {
  return getStateFromVoltage(gasVoltageFromPpm(ppm));
}

// Dashboard display state is voltage-based to match the ESP32 thresholds.
function getStateFromVoltage(voltage) {
  const value = Number(voltage);
  if (!Number.isFinite(value)) return 'safe';
  if (value >= VOLTAGE_THRESHOLDS.extreme) return 'extreme';
  if (value >= VOLTAGE_THRESHOLDS.danger)  return 'danger';
  if (value >= VOLTAGE_THRESHOLDS.warning) return 'warning';
  return 'safe';
}

const STATE_SEVERITY = { safe: 0, warning: 1, danger: 2, extreme: 3 };

// Normalises input from Firebase or mock data before severity comparison.
function normaliseStateName(state) {
  const value = String(state || '').trim().toLowerCase();
  return STATE_SEVERITY[value] === undefined ? 'safe' : value;
}

// Keeps the most severe state when the firmware state and dashboard voltage
// classification disagree, preventing a serious reading from being hidden.
function mostSevereState(...states) {
  return states
    .map(normaliseStateName)
    .sort((a, b) => STATE_SEVERITY[b] - STATE_SEVERITY[a])[0] || 'safe';
}

// Provides a voltage estimate for mock or legacy ppm-only readings.
function gasVoltageFromPpm(ppm) {
  const value = Number(ppm);
  if (!Number.isFinite(value) || value <= 0) return 0;

  if (value < PPM_THRESHOLDS.warning) {
    return (value / PPM_THRESHOLDS.warning) * VOLTAGE_THRESHOLDS.warning;
  }

  if (value < PPM_THRESHOLDS.danger) {
    const span = PPM_THRESHOLDS.danger - PPM_THRESHOLDS.warning;
    const pct = (value - PPM_THRESHOLDS.warning) / span;
    return VOLTAGE_THRESHOLDS.warning + pct * (VOLTAGE_THRESHOLDS.danger -
      VOLTAGE_THRESHOLDS.warning);
  }

  const span = PPM_THRESHOLDS.extreme - PPM_THRESHOLDS.danger;
  const pct = Math.min((value - PPM_THRESHOLDS.danger) / span, 1);
  return VOLTAGE_THRESHOLDS.danger + pct * (VOLTAGE_THRESHOLDS.extreme -
      VOLTAGE_THRESHOLDS.danger);
}

function formatVoltage(voltage) {
  const value = Number(voltage);
  return Number.isFinite(value) ? `${value.toFixed(2)} V` : '-- V';
}

function formatTime(date = new Date()) {
  return date.toLocaleTimeString('en-AU', {
    hour: '2-digit', minute: '2-digit', second: '2-digit', hour12: false,
  });
}

function formatDateTime(date = new Date()) {
  return date.toLocaleString('en-AU', {
    day: '2-digit', month: '2-digit',
    hour: '2-digit', minute: '2-digit', hour12: false,
  });
}

function getReadingDate(timestamp) {
  if (!timestamp) return new Date();
  const n = Number(timestamp);
  if (!Number.isFinite(n)) return new Date();
  return new Date(n < 10000000000 ? n * 1000 : n);
}

// -- UI -- status & metrics ---------------------------------------------------

const STATE_MESSAGES = {
  safe:    'System operating normally. Gas levels within safe range.',
  warning: 'Warning: elevated gas concentration detected. Monitor the area.',
  danger:  'DANGER: critical gas level detected. Immediate action required. Fan
      and buzzer active.',
  extreme: 'EXTREME: gas voltage is critically high. Evacuate and ventilate
      immediately.',
};

const VOLTAGE_SUBS = {
  safe:    'Safe: < 1.60 V',
  warning: 'Warning: >= 1.60 V',
  danger:  'Danger: >= 1.90 V',
  extreme: 'Extreme: >= 2.20 V',
};

const NOTIFICATION_TITLES = {
  warning: 'LeakSense warning',
  danger:  'LeakSense danger alert',
  extreme: 'LeakSense extreme alert',
};

let notificationRegistrationPromise = null;

// Registers the service worker once and reuses the ready registration for
// all later notifications.
function getNotificationRegistration() {
  if (!('serviceWorker' in navigator)) return Promise.resolve(null);

  if (!notificationRegistrationPromise) {
    notificationRegistrationPromise = navigator.serviceWorker
      .register('service-worker.js')
      .then(() => navigator.serviceWorker.ready);
  }

  return notificationRegistrationPromise;
}

// Enables browser notifications from a user click, as required by browser
    policy.
function setupAlertNotifications() {
  const btn = document.getElementById('enableNotificationsBtn');
  if (!btn || !('Notification' in window)) return;

  function syncButton() {
    if (Notification.permission === 'granted') {
      btn.hidden = false;
      btn.disabled = true;
      btn.textContent = 'Notifications on';
      return;
    }

    btn.hidden = false;
    btn.disabled = Notification.permission === 'denied';
    btn.textContent = Notification.permission === 'denied'
      ? 'Notifications blocked'
      : 'Enable notifications';
  }

  btn.addEventListener('click', async () => {
    try {
      const permission = await Notification.requestPermission();
      syncButton();

      if (permission === 'granted') {
        const registration = await getNotificationRegistration();
        await showLeakSenseNotification('LeakSense notifications enabled', {
          body: 'You will be notified when gas reaches warning, danger, or
      extreme levels.',
          tag: 'leaksense-notifications-enabled',
        }, registration);
      }
```

```
    } catch (err) {
      console.warn('Could not enable notifications:', err);
    }
  });

  getNotificationRegistration().catch(err => {
    console.warn('Could not register notification service worker:', err);
  });
  syncButton();
}

// Sends via service worker where possible so notifications still work when the
// page is in the background.
async function showLeakSenseNotification(title, options, registration = null) {
  const serviceWorkerRegistration = registration || await
      getNotificationRegistration();

  if (serviceWorkerRegistration && serviceWorkerRegistration.showNotification)
      {
    return serviceWorkerRegistration.showNotification(title, options);
  }

  return new Notification(title, options);
}

// Creates Warning, Danger, and Extreme notifications with stronger persistence
// for the more serious alert states.
function sendAlertNotification(voltage, state, thermalRisk = false) {
  if (!('Notification' in window) || Notification.permission !== 'granted')
      return;

  const title = NOTIFICATION_TITLES[state] || 'LeakSense alert';
  const body = thermalRisk
    ? `${formatVoltage(voltage)} detected with abnormal temperature rise. ${
      STATE_MESSAGES.danger}`
    : `${formatVoltage(voltage)} detected. ${STATE_MESSAGES[state]}`;

  showLeakSenseNotification(title, {
      body,
      tag: `leaksense-${state}`,
      renotify: true,
      requireInteraction: state === 'danger' || state === 'extreme',
    }).catch(err => {
    console.warn('Could not show notification:', err);
  });
}

// Updates the coloured status badge in the top bar.
function updateStatusBadge(state) {
  const el = document.getElementById('statusBadge');
  el.className  = `status-badge ${state}`;
  el.textContent = state.charAt(0).toUpperCase() + state.slice(1);
}

// Updates the full-width message bar below the header.
function updateStateBar(state) {
  const el = document.getElementById('stateBar');
  el.className  = `state-bar ${state}`;
  el.textContent = STATE_MESSAGES[state];
}

// Updates the four top metric cards from the latest reading.
function updateMetrics(data) {
  document.getElementById('ppmVal').textContent  = formatVoltage(data.voltage);
  document.getElementById('ppmSub').textContent  = data.thermal_risk
    ? 'Danger: gas + thermal risk'
    : VOLTAGE_SUBS[data.state];
  document.getElementById('tempVal').textContent = `${data.temperature}°C`;
  document.getElementById('humVal').textContent  = `${data.humidity}%`;
  document.getElementById('lastSync').textContent = `Last sync: ${formatTime(
      getReadingDate(data.timestamp))}`;
}

// -- UI -- device states with yellow LED ------------------------------------
      ---

function updateDeviceStates(data) {
  // Fan -- on in Danger/Extreme by default from Firebase normalization
  const fan = document.getElementById('fanState');
  fan.textContent = data.fan ? 'on' : 'off';
  fan.className   = `pill${data.fan ? ' pill--on' : ''}`;

  // Buzzer -- on in Danger/Extreme by default from Firebase normalization
  const buz = document.getElementById('buzzerState');
  buz.textContent = data.buzzer ? 'on' : 'off';
  buz.className   = `pill${data.buzzer ? ' pill--on' : ''}`;

  // Green LED -- Safe only
  const green = document.getElementById('greenLed');
  green.textContent = data.state === 'safe' ? 'on' : 'off';
  green.className   = `pill${data.state === 'safe' ? ' pill--on' : ''}`;

  // Yellow LED -- Warning only
  const yellow = document.getElementById('yellowLed');
  yellow.textContent = data.state === 'warning' ? 'on' : 'off';
  yellow.className   = `pill${data.state === 'warning' ? ' pill--warning' :
      ''}`;

  // Red LED -- Danger and Extreme
  const red = document.getElementById('redLed');
  const redActive = data.state === 'danger' || data.state === 'extreme';
  red.textContent = redActive ? 'on' : 'off';
  red.className   = `pill${redActive ? ` pill--${data.state}` : ''}`;
}

// -- UI -- alert log --------------------------------------------------------
      ----

function addAlertEntry(voltage, state, thermalRisk = false) {
  const time = formatTime();
  const formattedVoltage = formatVoltage(voltage);
  const labels = {
    warning: 'Warning threshold crossed',
    danger:  'DANGER threshold crossed',
    extreme: 'EXTREME threshold crossed',
  };
  const msg = thermalRisk
    ? `Gas reached ${formattedVoltage} -- thermal risk escalation`
    : `Gas reached ${formattedVoltage} -- ${labels[state] || 'Alert threshold
       crossed'}`;

  alertLog.unshift({ time, voltage, state, msg });
  if (alertLog.length > 20) alertLog.pop();

  alertCount++;
  document.getElementById('alertCount').textContent = alertCount;
  renderAlertLog();
}

function renderAlertLog() {
  const el = document.getElementById('alertLog');
  if (alertLog.length === 0) {
    el.innerHTML = '<p class="alert-log__empty">No alerts yet -- system safe.</
       p>';
    return;
  }
  el.innerHTML = alertLog.map(a => `
    <div class="alert-row">
      <span class="alert-row__time">${a.time}</span>
      <span class="alert-row__msg">${a.msg}</span>
      <span class="alert-pill alert-pill--${a.state}">${a.state.toUpperCase()
       }</span>
    </div>
  `).join('');
}

// -- UI -- 24hr history table -----------------------------------------------
       ----

/**
* Adds a history reading to the table.
* Prepends newest rows so most recent is always at the top.
* @param {object} reading - normalised reading
*/
function addHistoryRow(reading) {
  const date = getReadingDate(reading.timestamp);
  historyRows.unshift({
    time:  formatDateTime(date),
    voltage: reading.voltage,
    temp:  reading.temperature,
    hum:   reading.humidity,
    state: reading.state,
  });

  // Trim to 24hr window
  if (historyRows.length > MAX_HISTORY_ROWS) historyRows.pop();

  renderHistoryTable();
}

/**
* Replaces table with full history array (initial load).
* @param {Array} readings - array of normalised readings oldest→newest
*/
function populateHistoryTable(readings) {
  historyRows.length = 0;
  // Reverse so newest is first
  [...readings].reverse().forEach(r => {
    const date = getReadingDate(r.timestamp);
    historyRows.push({
      time:  formatDateTime(date),
      voltage: r.voltage,
      temp:  r.temperature,
      hum:   r.humidity,
      state: r.state,
    });
  });
  renderHistoryTable();
}

function stateLabel(state) {
  const map = {
    safe:    '<span class="hist-badge hist-badge--safe">Safe</span>',
    warning: '<span class="hist-badge hist-badge--warning">Warning</span>',
    danger:  '<span class="hist-badge hist-badge--danger">Danger</span>',
    extreme: '<span class="hist-badge hist-badge--extreme">Extreme</span>',
  };
  return map[state] || state;
}

function renderHistoryTable() {
  const tbody = document.getElementById('historyTableBody');
  const empty = document.getElementById('historyEmpty');

  if (historyRows.length === 0) {
    tbody.innerHTML = '';
    if (empty) empty.style.display = 'block';
    return;
  }

  if (empty) empty.style.display = 'none';

  tbody.innerHTML = historyRows.map(r => `
    <tr>
      <td>${r.time}</td>
      <td><strong>${formatVoltage(r.voltage)}</strong></td>
      <td>${r.temp.toFixed(1)}°C</td>
      <td>${r.hum.toFixed(1)}%</td>
      <td>${stateLabel(r.state)}</td>
    </tr>
```

```
  `).join('');
}

// -- UI -- live/history view switch ------------------------------------------
function setupDashboardViews() {
  const liveView = document.getElementById('liveView');
  const historyView = document.getElementById('historyView');
  const showHistoryBtn = document.getElementById('showHistoryBtn');
  const showLiveBtn = document.getElementById('showLiveBtn');

  function showView(view) {
    const showingHistory = view === 'history';
    liveView.hidden = showingHistory;
    historyView.hidden = !showingHistory;

    if (showingHistory && historyChart) {
      historyChart.resize();
      historyChart.update('none');
    }

    if (!showingHistory && trendChart) {
      trendChart.resize();
      trendChart.update('none');
    }
  }

  showHistoryBtn.addEventListener('click', () => showView('history'));
  showLiveBtn.addEventListener('click', () => showView('live'));
  showView('live');
}

// -- Main update -- called on every live reading -----------------------------
// Single entry point for live data. Every Firebase or mock reading flows through
// here before updating badges, metrics, device states, charts, alerts, and notifications.
function onNewReading(data) {
  const time = formatTime(getReadingDate(data.timestamp));
  const voltage = Number(data.voltage);
  if (!Number.isFinite(voltage)) {
    data.voltage = gasVoltageFromPpm(data.ppm_compensated);
  }
  data.state = mostSevereState(data.state, getStateFromVoltage(data.voltage));

  updateStatusBadge(data.state);
  updateStateBar(data.state);
  updateMetrics(data);
  updateDeviceStates(data);
  updateChart(data.voltage, time);  // charts.js

  if (data.state !== 'safe' && data.state !== prevState) {
    addAlertEntry(data.voltage, data.state, data.thermal_risk);
    sendAlertNotification(data.voltage, data.state, data.thermal_risk);
  }

  prevState = data.state;
}

// -- Initialise --------------------------------------------------------------
document.addEventListener('DOMContentLoaded', () => {
  // Initialise visual components before attaching Firebase listeners so the
  // first incoming reading has a chart and DOM to update.
  initChart();         // live chart -- charts.js
  initHistoryChart();  // 24hr chart -- charts.js
  setupDashboardViews();
  setupAlertNotifications();

  if (typeof listenToSensorData === 'function' && listenToSensorData(onNewReading)) {
    // Live readings → dashboard

    // 24hr history → chart + table (initial load)
    loadHistory(readings => {
      populateHistoryChart(readings);   // charts.js
      populateHistoryTable(readings);   // dashboard.js
    });

    // Stream new history entries in real time
    listenToHistory(reading => {
      addHistoryPoint(reading);  // charts.js
      addHistoryRow(reading);    // dashboard.js
    });

    return;
  }

  // -- Mock fallback ---------------------------------------------------------
  onNewReading(getMockReading());
  setInterval(() => {
    const reading = getMockReading();
    onNewReading(reading);
  }, 2000);
});
```

Listing 9. Chart JavaScript: dashboard/js/charts.js.

```
/**
* charts.js
* Manages two Chart.js charts:
*  - trendChart  : live readings, last 60 points
*  - historyChart: 24-hour voltage + temperature + humidity
*/

const THRESHOLD_WARNING_V = 1.60;
const THRESHOLD_DANGER_V  = 1.90;
const THRESHOLD_EXTREME_V = 2.20;
const GAS_AXIS_MAX_V      = 3.00; // max voltage shown on y-axis, for better visual scaling
const MAX_LIVE_POINTS   = 60;

let trendChart   = null;
let historyChart = null;

// -- Live trend chart --------------------------------------------------------
// Creates the live chart shown on the main dashboard. Three flat threshold
// datasets are drawn beside the gas series for visual comparison.
function initChart() {
  const ctx = document.getElementById('trendChart').getContext('2d');

  trendChart = new Chart(ctx, {
    type: 'line',
    data: {
      labels:   [],
      datasets: [
        {
          label:            'Gas (V)',
          data:             [],
          borderColor:      '#3b82f6',
          backgroundColor:  'rgba(59,130,246,0.07)',
          borderWidth:      2,
          pointRadius:      2,
          pointHoverRadius: 4,
          tension:          0.4,
          fill:             true,
        },
        {
          label:       'Warning (1.60 V)',
          data:        [],
          borderColor: 'rgba(217,119,6,0.6)',
          borderWidth: 1,
          borderDash:  [5,4],
          pointRadius: 0,
          fill:        false,
        },
        {
          label:       'Danger (1.90 V)',
          data:        [],
          borderColor: 'rgba(185,28,28,0.6)',
          borderWidth: 1,
          borderDash:  [5,4],
          pointRadius: 0,
          fill:        false,
        },
        {
          label:       'Extreme (2.20 V)',
          data:        [],
          borderColor: 'rgba(127,29,29,0.6)',
          borderWidth: 1,
          borderDash:  [5,4],
          pointRadius: 0,
          fill:        false,
        },
      ],
    },
    options: {
      responsive:          true,
      maintainAspectRatio: false,
      animation:           { duration: 300 },
      interaction:         { mode: 'index', intersect: false },
      plugins: {
        legend: { display: false },
        tooltip: {
          callbacks: {
            label: ctx => ctx.datasetIndex === 0 ? `${Number(ctx.raw).toFixed(2)} V` : null,
          },
        },
      },
      scales: {
        x: {
          display: true,
          ticks:   { font: { size: 10 }, color: '#9ca3af', maxTicksLimit: 6 },
          grid:    { display: false },
          border:  { display: false },
        },
        y: {
          min:    0,
          max:    GAS_AXIS_MAX_V,
          ticks:  { font: { size: 10 }, color: '#9ca3af', maxTicksLimit: 6 },
          grid:   { color: '#f3f4f6' },
          border: { display: false },
        },
      },
    },
  });
}

// Appends one live voltage sample and trims the chart to the latest 60 readings.
function updateChart(voltage, time) {
  const [gasData, warnData, dangerData, extremeData] = trendChart.data.datasets.map(d => d.data);
  const labels = trendChart.data.labels;

  gasData.push(voltage);
  warnData.push(THRESHOLD_WARNING_V);
  dangerData.push(THRESHOLD_DANGER_V);
  extremeData.push(THRESHOLD_EXTREME_V);
  labels.push(time);

  if (gasData.length > MAX_LIVE_POINTS) {
    gasData.shift(); warnData.shift(); dangerData.shift(); extremeData.shift(); labels.shift();
```

```
  }

  trendChart.update('none');
}

// Clears all live chart points; useful during manual testing or future resets.
function resetChart() {
  trendChart.data.labels = [];
  trendChart.data.datasets.forEach(d => { d.data = []; });
  trendChart.update('none');
}

// -- 24-hour history chart ---------------------------------------------------

// Creates the history chart with two axes: gas voltage on the left and
// temperature/humidity on the right.
function initHistoryChart() {
  const ctx = document.getElementById('historyChart').getContext('2d');

  historyChart = new Chart(ctx, {
    type: 'line',
    data: {
      labels:   [],
      datasets: [
        {
          label:           'Gas (V)',
          data:            [],
          borderColor:     '#3b82f6',
          backgroundColor: 'rgba(59,130,246,0.08)',
          borderWidth:     2,
          pointRadius:     2,
          pointHoverRadius: 5,
          tension:         0.35,
          fill:            true,
          yAxisID:         'yGas',
        },
        {
          label:           'Temp (°C)',
          data:            [],
          borderColor:     '#f59e0b',
          backgroundColor: 'transparent',
          borderWidth:     1.5,
          pointRadius:     0,
          tension:         0.35,
          fill:            false,
          yAxisID:         'yEnv',
        },
        {
          label:           'Humidity (%)',
          data:            [],
          borderColor:     '#10b981',
          backgroundColor: 'transparent',
          borderWidth:     1.5,
          pointRadius:     0,
          tension:         0.35,
          fill:            false,
          yAxisID:         'yEnv',
        },
        {
          label:       'Warning (1.60 V)',
          data:        [],
          borderColor: 'rgba(217,119,6,0.5)',
          borderWidth: 1,
          borderDash:  [5,4],
          pointRadius: 0,
          fill:        false,
          yAxisID:     'yGas',
        },
        {
          label:       'Danger (1.90 V)',
          data:        [],
          borderColor: 'rgba(185,28,28,0.5)',
          borderWidth: 1,
          borderDash:  [5,4],
          pointRadius: 0,
          fill:        false,
          yAxisID:     'yGas',
        },
        {
          label:       'Extreme (2.20 V)',
          data:        [],
          borderColor: 'rgba(127,29,29,0.5)',
          borderWidth: 1,
          borderDash:  [5,4],
          pointRadius: 0,
          fill:        false,
          yAxisID:     'yGas',
        },
      ],
    },
    options: {
      responsive:          true,
      maintainAspectRatio: false,
      animation:           { duration: 200 },
      interaction:         { mode: 'index', intersect: false },
      plugins: {
        legend: {
          display: true,
          position: 'top',
          labels: { font: { size: 11 }, color: '#6b7280', boxWidth: 14, padding
        : 12 },
        },
        tooltip: {
          callbacks: {
            label: ctx => {
              if (ctx.datasetIndex === 0) return `Gas: ${Number(ctx.raw).
        toFixed(2)} V`;
              if (ctx.datasetIndex === 1) return `Temp: ${ctx.raw.toFixed(1)}°
        C`;
              if (ctx.datasetIndex === 2) return `Humidity: ${ctx.raw.toFixed
        (1)}%`;
              return null;
            },
          },
        },
      },
      scales: {
        x: {
          display: true,
          ticks:   { font: { size: 10 }, color: '#9ca3af', maxTicksLimit: 8 },
          grid:    { display: false },
          border:  { display: false },
        },
        yGas: {
          type:     'linear',
          position: 'left',
          min:      0,
          max:      GAS_AXIS_MAX_V,
          title:    { display: true, text: 'Gas (V)', color: '#3b82f6', font: {
        size: 11 } },
          ticks:    { font: { size: 10 }, color: '#9ca3af' },
          grid:     { color: '#f3f4f6' },
          border:   { display: false },
        },
        yEnv: {
          type:     'linear',
          position: 'right',
          min:      0,
          max:      100,
          title:    { display: true, text: 'Temp °C / Humidity %', color: '#6
        b7280', font: { size: 11 } },
          ticks:    { font: { size: 10 }, color: '#9ca3af' },
          grid:     { drawOnChartArea: false },
          border:   { display: false },
        },
      },
    },
  });
}

/**
* Adds a single history reading to the 24hr chart.
* Called both when loading existing history and on new live entries.
* @param {object} reading - normalised reading from firebase.js
*/
function addHistoryPoint(reading) {
  const date   = new Date(reading.timestamp);
  const label  = date.toLocaleTimeString('en-AU', { hour: '2-digit', minute:
       '2-digit', hour12: false });

  const [gasDs, tempDs, humDs, warnDs, dangerDs, extremeDs] = historyChart.data
       .datasets;

  gasDs.data.push(reading.voltage);
  tempDs.data.push(reading.temperature);
  humDs.data.push(reading.humidity);
  warnDs.data.push(THRESHOLD_WARNING_V);
  dangerDs.data.push(THRESHOLD_DANGER_V);
  extremeDs.data.push(THRESHOLD_EXTREME_V);
  historyChart.data.labels.push(label);

  historyChart.update('none');
}

/**
* Replaces all history chart data at once (used on initial load).
* @param {Array} readings - array of normalised reading objects
*/
function populateHistoryChart(readings) {
  historyChart.data.labels = [];
  historyChart.data.datasets.forEach(d => { d.data = []; });

  readings.forEach(r => addHistoryPoint(r));
}
```

Listing 10. Mock data JavaScript: dashboard/js/mock.js.

```
/**
 * mock.js
 * Simulates ESP32 sensor data for development.
 * Remove this file (and the script tag in index.html) when Firebase is
      connected.
 * Replace with firebase.js instead.
 */

// Used only as a local fallback when Firebase is unavailable.
let mockBasePpm = 120;

/**
 * Adds realistic noise to a base value -- mimics real sensor jitter
 * @param {number} base
 * @returns {number}
 */
function addNoise(base) {
  return Math.max(0, Math.round(base + (Math.random() - 0.5) * 20));
}

/**
 * Returns a simulated sensor reading matching the exact
 * same structure the ESP32 will push to Firebase.
 * Field names must match firebase.js and the ESP32 firmware.
 *
 * @returns {{
 *   ppm_compensated: number,
 *   ppm_raw: number,
 *   temperature: number,
 *   humidity: number,
 *   state: string,
 *   fan: boolean,
 *   buzzer: boolean,
```

```
 *   timestamp: number
 * }}
 */
function getMockReading() {
  const ppm   = addNoise(mockBasePpm);
  const state = getStateFromPpm(ppm);

  // Keep the mock payload aligned with the ESP32/Firebase payload so the same
  // dashboard code is exercised during offline development.
  return {
    ppm_compensated: ppm,
    ppm_raw:         Math.round(ppm * 1.08),  // raw is slightly higher before compensation
    temperature:     parseFloat((22 + Math.random() * 2).toFixed(1)),
    humidity:        parseFloat((45 + Math.random() * 5).toFixed(1)),
    state:           state,
    thermal_risk:    false,
    fan:             state !== 'safe',
    buzzer:          state !== 'safe',
    timestamp:       Date.now(),
  };
}

/**
 * Allows dashboard.js to update the mock base ppm via the slider
 * @param {number} val
 */
function setMockBasePpm(val) {
  mockBasePpm = val;
}
```

Listing 11. Notification service worker: dashboard/service-worker.js.

```
// Handles clicks on browser push notifications generated by dashboard.js.
self.addEventListener('notificationclick', event => {
  event.notification.close();

  event.waitUntil((async () => {
    // Reuse an already-open dashboard tab when possible; otherwise open one.
    const windows = await clients.matchAll({ type: 'window', includeUncontrolled: true });
    const openWindow = windows.find(client => client.url.includes(self.location.origin));

    if (openWindow) {
      await openWindow.focus();
      return;
    }

    await clients.openWindow('/');
  })());
});
```